



evatronix

We turn engineering into art

Synthesizable HDL Core Specification

Research and Development Department

Poland, 44-100 Gliwice, Dubois 16

Phone/Fax: +48 32 231 11 71, +48 32 231 30 27

e-mail: ipcenter@evatronix.com.pl

url: www.evatronix.pl

August 16, 2004

Gigabit Ethernet

Media Access Controller

With AMBA Interface

Synthesizable HDL Core Specification

Version 2.02

Contents

1	INTRODUCTION.....	5
1.1	OVERVIEW	5
1.2	FEATURES	6
1.3	VERSION HISTORY	6
1.4	SUPPLIED FILES	7
2	HARDWARE DESCRIPTION	14
2.1	SYMBOL	14
2.2	BLOCK DIAGRAM	15
2.3	PIN DESCRIPTION.....	19
2.4	CONTROL AND STATUS REGISTERS (CSRs)	22
2.5	AHB INTERFACE.....	53
2.6	FRAME DESCRIPTORS	56
2.7	EXTERNAL DUAL-PORT RAM INTERFACE	67
2.8	STATISTICAL COUNTERS CONTROLLER.....	70
2.9	DMA CONTROLLER	73
2.10	TRANSMIT PROCESS	74
2.11	RECEIVE PROCESS	76
2.12	INTERRUPT CONTROL	78
2.13	GENERAL PURPOSE TIMER.....	80
2.14	DATA LINK LAYER OPERATION.....	81
2.15	CLOCK CONTROL.....	89
2.16	RESET CONTROL.....	90
3	USER'S GUIDE.....	91
3.1	BASIC DESIGN FLOW.....	91
3.2	COMPILATION NOTES	92
3.3	TEST BENCH DESCRIPTION	95
3.4	SIMULATION NOTES.....	101
4	SOURCES.....	103

Figures

FIGURE 1. BLOCK DIAGRAM	15
FIGURE 2. APB WRITE OPERATIONS	24
FIGURE 3. APB READ OPERATIONS	24
FIGURE 2. THE BUS MODE REGISTER	25
FIGURE 3. THE TRANSMIT POLL DEMAND REGISTER	27
FIGURE 4. THE RECEIVE POLL DEMAND REGISTER	28
FIGURE 5. THE RECEIVE DESCRIPTOR LIST BASE ADDRESS REGISTER	29
FIGURE 6. THE TRANSMIT DESCRIPTOR LIST BASE ADDRESS REGISTER	29
FIGURE 7. THE STATUS REGISTER	30
FIGURE 8. THE OPERATION MODE REGISTER	33
FIGURE 9. THE INTERRUPT ENABLE REGISTER	37
FIGURE 10. THE MISSED FRAMES AND OVERFLOW COUNTER REGISTER	39
FIGURE 11. THE SERIAL ROM REGISTER	40
FIGURE 12. EXTERNAL SERIAL ROM CONNECTIONS	41
FIGURE 13. THE GMII/MII MANAGEMENT REGISTER	42
FIGURE 14. TYPICAL MII SERIAL MANAGEMENT READ OPERATION	43
FIGURE 15. EXTERNAL TRISTATE BUFFER CONNECTIONS	43
FIGURE 16. THE GENERAL-PURPOSE TIMER AND INTERRUPT MITIGATION CONTROL REGISTER	45
FIGURE 17. MAC ADDRESS LOW REGISTER	46
FIGURE 18. MAC ADDRESS HIGH REGISTER	47
FIGURE 19. PAUSE TIME & CACHE THRESHOLDS REGISTER	48
FIGURE 20. PAUSE TIME & CACHE THRESHOLDS REGISTER	49
FIGURE 21. FLOW CONTROL SETUP & STATUS	50
FIGURE 22. SC MODE REGISTER	51
FIGURE 23. SC READ VALUE REGISTER	52
FIGURE 24. AHB SINGLE WRITE OPERATION	53
FIGURE 16. AHB BURST WRITE OPERATION	53
FIGURE 17. AHB SINGLE READ OPERATION	54
FIGURE 18. AHB BURST READ OPERATION	54
FIGURE 25. DESCRIPTORS IN "RING STRUCTURE"	57
FIGURE 26. DESCRIPTORS IN "CHAINED STRUCTURE"	58
FIGURE 27. RECEIVE DESCRIPTOR FIELDS	59
FIGURE 28. TRANSMIT DESCRIPTOR FIELDS	62
FIGURE 29. DUAL PORT RAM WRITE AND READ OPERATIONS	67
FIGURE 30. SC DUAL PORT RAM WRITE AND READ OPERATIONS	69
FIGURE 31. TRANSMIT PROCESS TRANSITIONS	75
FIGURE 32. RECEIVE PROCESS TRANSITIONS	77
FIGURE 33. INTERRUPT SCHEME	79
FIGURE 34. GMII/MII RECEIVE OPERATION	82
FIGURE 35. GMII/MII TRANSMIT OPERATION	83
FIGURE 36. 802.3 FRAME FORMAT	84
FIGURE 37. BACKOFF PROCESS ALGORITHM	86
FIGURE 38. DEFERENCE PROCESS ALGORITHM	87
FIGURE 39. FILTERING WITH 1 PHYSICAL ADDRESS AND THE HASH TABLE	88
FIGURE 40. CLOCK DOMAINS	89
FIGURE 41. TEST BENCH BLOCK DIAGRAM	96

Tables

TABLE 1. PIN DESCRIPTION	19
TABLE 2. CONTROL AND STATUS REGISTERS LOCATIONS	22
TABLE 3. CONTROL AND STATUS REGISTER APB INTERFACE SIGNALS	23
TABLE 4. THE BUS MODE REGISTER BIT FUNCTIONS	25
TABLE 5. TRANSMIT AUTOMATIC POLLING INTERVALS	26
TABLE 6. THE TRANSMIT POLL DEMAND BIT FUNCTIONS	27
TABLE 7. THE RECEIVE POLL DEMAND BIT FUNCTIONS	28
TABLE 8. THE RECEIVE DESCRIPTOR LIST BASE ADDRESS REGISTER BIT FUNCTIONS	29
TABLE 9. THE TRANSMIT DESCRIPTOR LIST BASE ADDRESS REGISTER BIT FUNCTIONS	29
TABLE 10. THE STATUS REGISTER BIT FUNCTIONS	30
TABLE 11. THE OPERATION MODE REGISTER BIT FUNCTIONS	33
TABLE 12. RECEIVE ADDRESS FILTERING MODES SUMMARY	35
TABLE 13. TRANSMIT FIFO THRESHOLD LEVELS (BYTES)	36
TABLE 14. THE INTERRUPT ENABLE REGISTER BIT FUNCTIONS	37
TABLE 15. THE MISSED FRAMES AND OVERFLOW COUNTER BIT FUNCTIONS	39
TABLE 16. THE SERIAL ROM REGISTER BIT FUNCTIONS	40
TABLE 17. MIISM FRAME FORMAT	43
TABLE 18. GMII/MII MANAGEMENT REGISTER BIT FUNCTIONS	44
TABLE 19. THE GENERAL-PURPOSE TIMER AND INTERRUPT MITIGATION CONTROL BIT FUNCTIONS	45
TABLE 20. THE MAC ADDRESS LOW REGISTER BIT FUNCTIONS	46
TABLE 21. THE MAC ADDRESS LOW REGISTER BIT FUNCTIONS	47
TABLE 22. PAUSE TIME & CACHE THRESHOLDS REGISTER BIT FUNCTIONS	48
TABLE 23. PAUSE TIME & CACHE THRESHOLDS REGISTER BIT FUNCTIONS	49
TABLE 24. FLOW CONTROL SETUP & STATUS BIT FUNCTIONS	50
TABLE 25. SC MODE REGISTER BIT FUNCTIONS	51
TABLE 26. SC READ VALUE BIT FUNCTIONS	52
TABLE 27. DATA INTERFACE SIGNALS	55
TABLE 28. STATUS (RDES0) BIT FUNCTIONS	59
TABLE 29. CONTROL AND COUNT (RDES1) BIT FUNCTIONS	61
TABLE 30. RBA1 (RDES2) BIT FUNCTIONS	61
TABLE 31. RBA2 (RDES3) BIT FUNCTIONS	61
TABLE 32. STATUS (TDES0) BIT FUNCTIONS	62
TABLE 33. CONTROL (TDES1) BIT FUNCTIONS	63
TABLE 34. TBA1 (TDES2) BIT FUNCTIONS	64
TABLE 35. TBA2 (TDES3) BIT FUNCTIONS	64
TABLE 36. FILTERING TYPE SELECTION	65
TABLE 37. PERFECT FILTERING SETUP FRAME BUFFER FORMAT	65
TABLE 38. HASH TABLE SETUP FRAME BUFFER FORMAT	66
TABLE 39. DUAL PORT RAM PINS MAPPING	67
TABLE 40. DUAL-PORT RAM PARAMETERS	68
TABLE 41. SC DUAL PORT RAM PINS MAPPING	69
TABLE 42. TRANSMIT STATISTICAL COUNTERS DESCRIPTION	71
TABLE 43. RECEIVE STATISTICAL COUNTERS DESCRIPTION	72
TABLE 44. EXTERNAL PHY INTERFACE PINS	81
TABLE 45. FRAME FIELDS USAGE	84

1 Introduction

This manual is intended for design engineers who are planning to use the MAC-1G_AMBA HDL core in conjunction with simulation and synthesis tools.

1.1 Overview

The MAC-1G_AMBA Ethernet controller is a synthesizable HDL core of a high-speed LAN controller. It implements Carrier Sense Multiple Access with Collision Detection (CSMA/CD) algorithms defined by IEEE 802.3 for media access control over the 10Mbps, 100Mbps and 1Gbps Ethernet.

Communication with an external host is implemented via a set of Control and Status Registers accessed via AMBA™ Advanced Peripheral Bus interface and the AMBA™ Advanced High-performance Bus controller for access to external shared RAM memory. For data transfers the MAC-1G_AMBA operates as a DMA master. It automatically fetches from transmit data buffers and stores receive data buffers into external RAM with minimum CPU intervention. The linked list management enables the use of various memory allocation schemes. There is an interface for external dual port RAMs serving as configurable FIFO memories and there are separate memories for transmit and receive processes. Using the FIFOs additionally isolates the MAC-1G_AMBA from an external host and provides resolution in case of latency of an external bus.

1.2 Features

- Network interface features
 - Supports 10/100/1000 Mb/s data transfer rate
 - Gigabit Media Independent Interface (GMII) for 1Gbps operation
 - Media Independent Interface (MII) for 10/100 Mbps operation
- Data link layer functionality
 - Meets the IEEE 802.3 CSMA/CD standard
 - Full or half duplex operation
 - Flexible address filtering
 - External RAM for storing MAC addresses
 - Up to 16 physical addresses
 - 512 bit hash table for multicast addresses
- Control and status registers
 - Configurable 8/16/32 bit APB slave interface
 - Single interrupt line
 - Interrupt mitigation control mechanism
- DMA controller
 - Configurable 8/16/32 bit AHB interface
 - Configurable address bus length
 - Big or little endian data byte ordering
 - Scatter/gather capabilities
 - Programmable burst length
 - Intelligent arbitration between transmit and receive processes
- Descriptor/buffer architecture for data storage
 - Descriptor "ring" or "chain" structures
 - Single descriptor points to up to two data buffers
 - Automatic descriptor list pooling
- Low power capabilities
 - Independent clocks for data and control paths
 - Running/Suspended/Stopped modes of operation
 - Clock switching support
- Transmit/Receive dual port RAM interfaces
 - Operates as internal configurable FIFOs
 - Programmable threshold levels
 - "Store and forward" functionality
- Applications
 - Network Interface Cards (NICs)
 - Routers, switching hubs
 - Systems On Chip (SoCs)

1.3 Version history

Version 1.00 – December 2002

Version 2.00 – November 2003

Version 2.02 – August 2004

1.4 Supplied files

1.4.1 Documentation files

- a) Specification and Data Sheet files:
MAC_1G_AMBA/doc/
mac_1g_amba_spec.pdf : This document file
mac_1g_amba_tests.pdf : Test set details

1.4.2 Source code files

- a) Primary source code files for MAC-1G_AMBA (synthesizable):
MAC_1G_AMBA/src/core/
utility_mac_1g.vhd (or utility_mac_1g.v) : Package with Constants
tc.vhd (or tc.v) : Transmit Controller
rc.vhd (or rc.v) : Receive Controller
bd.vhd (or bd.v) : Backoff/Deffering
tfifo.vhd (or tfifo.v) : Transmit FIFO
rfifo.vhd (or rfifo.v) : Receive FIFO
tlsm.vhd (or tlsm.v) : Transmit Linked List
rlsm.vhd (or rlsm.v) : Receive Linked List
dma.vhd (or dma.v) : Direct Memory Access
csr.vhd (or csr.v) : Control and Status Registers
rstc.vhd (or rstc.v) : Reset controller
miism.vhd (or miism.v) : GMII/MII Management
sc.vhd (or sc.v) : Statistical counters controller
fcr.vhd (or fcr.v) : Receive flow control
fct.vhd (or fct.v) : Transmit flow control
bp.vhd (or bp.v) : Back pressure control
festat.vhd (or festat.v) : Flow control statistical info
dom.vhd (or dom.v) : Clock domain synchronizer
domreq.vhd (or domreq.v) : Synchronizer request
domack.vhd (or domack.v) : Synchronizer acknowledge
mac_1g.vhd (or mac_1g.v) : RTL core of the MAC-1G
- b) Primary source code files for MAC2AMBA wrapper (synthesizable):
MAC_1G_AMBA/src/core/mac2amba
mac2amba_package.vhd (or mac2amba_package.v) : Package with constants
macdata2ahb.vhd (or macdata2ahb.v) : AHB wrapper
maccsr2apb.vhd (or maccsr2apb.v) : APB wrapper
mac2amba.vhd (or mac2amba.v) : RTL core of MAC2AMBA

1.4.3 Test Bench related files

a) Supplementary source code files:

MAC_1G_AMBA/src/tb/chip

tris.vhd	(or tris.v)	: Tristate buffer
dualram.vhd	(or dualram.v)	: Internal dual port RAM
scdram.vhd	(or scdram.v)	: Internal dual port RAM
chip_mac_1g_amba.vhd	(or chip_mac_1g_amba.v)	: MAC-1G_AMBA chip

b) Environment related files:

MAC_1G_AMBA/src/tb/env

clkgen.vhd	(or clkgen.v)	: External clock generator
rstgen.vhd	(or rstgen.v)	: External reset generator
linkmon.vhd	(or linkmon.v)	: GMII/MII monitor
smemstimahb.vhd	(or smemstimahb.v)	: System memory model
apbcmd.vhd	(or apbcmd.v)	: APB interface monitor
ambaarbcmd.vhd	(or ambaarbcmd.v)	: AMBA arbiter model
cam.vhd	(or cam.v)	: CAM memory model
intmon.vhd	(or intmon.v)	: Interrupt line monitor

c) Test Bench related files:

MAC_1G_AMBA/src/tb/

mac_1g_amba_tb.vhd	(or mac_1g_amba_tb.v)	: Test Bench
--------------------	-----------------------	--------------

d) Tests support files

MAC_1G_AMBA/tests/

default/	: Directory with default test file
d08/	: tests for 8-bit version
d08_fc/	: tests for 8-bit version flow control
d08_sc/	: tests for 8-bit version statistical counters
d08_g/	: tests for 8-bit version in 1G mode
d08_gsc/	: tests for 8-bit version statistical counters in 1G mode
d16/	: tests for 16-bit version
d16_fc/	: tests for 16-bit version flow control
d16_sc/	: tests for 16-bit version statistical counters
d16_g/	: tests for 16-bit version in 1G mode
d16_gsc/	: tests for 16-bit version statistical counters in 1G mode
d32/	: tests for 32-bit version
d32_fc/	: tests for 32-bit version flow control
d32_sc/	: tests for 32-bit version statistical counters
d32_g/	: tests for 32-bit version in 1G mode
d32_gsc/	: tests for 32-bit version statistical counters in 1G mode

Each test is composed of twelve files:

apbstim.txt	: Stimulus file for APB interface
linkstim.txt	: Stimulus file for GMII/MII interface
smemstim.txt	: Stimulus file for System RAM
arbstim.txt	: Stimulus file for arbiter
apbcomp.txt	: Expected Test Bench results from APB interface
ahbcomp.txt	: Expected Test Bench results from AHB interface
linkcomp.txt	: Expected Test Bench results from GMII/MII
intcomp.txt	: Expected Test Bench results from interrupt line
apbdiff.txt	: APB interface difference file
ahbdiff.txt	: AHB interface difference file
linkdiff.txt	: GMII/MII interface difference file
intdiff.txt	: Interrupt line difference file
time.txt	: Proposed simulation time
generic.txt	: Proposed set of generic parameters

1.4.4 MTI Modelsim EE support files

MAC_1G_AMBA/tools/mti/ee_macros/ - contains simulation macros and script files for ModelTech simulation tool

- compile.do – MTI macro file, it creates all needed libraries and compiles all source files
- simulate.do – MTI macro file, it initializes the simulation with the test case placed in the *test/default* directory, and then it creates a waveform with all vital signals from the MAC_1G_AMBA core and the Test Bench.
- readme.txt – A short description how to use both macro files.

MAC_1G_AMBA/tools/mti/ee_scripts/ - contains a set of UNIX shell (bash) scripts, DOS bat files and TCL script files for design compilation, and run for every test group.

- 08bit_tests – **TCL** script file. It invokes the MTI simulator for every test case in the /tests/d08* directories. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not.
 - **UNIX** shell file. This script file invokes the 08bit_tests.tcl file, which allows running the 8 bit test group.
 - **DOS** bat file. This script file invokes the 08bit_tests.tcl file, which allows running the 8 bit test group.
- 16bit_tests – **TCL** script file. It invokes the MTI simulator for every test case in the /tests/d16* directories. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not.
 - **UNIX** shell file. This script file invokes the 16bit_tests.tcl file, which allows running the 16 bit test group.
 - **DOS** bat file. This script file invokes the 16bit_tests.tcl file, which allows running the 16 bit test group.
- 32bit_tests – **TCL** script file. It invokes the MTI simulator for every test case in the /tests/d32* directories. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not.
 - **UNIX** shell file. This script file invokes the 32bit_tests.tcl file, which allows running the 32 bit test group.
 - **DOS** bat file. This script file invokes the 32bit_tests.tcl file, which allows running the 32 bit test group.

MAC_1G_AMBA/tools/mti/ee_reports/ - contains simulation reports files.

1.4.5 Aldec ActiveHDL support files

- MAC_1G_MAC/tools/aldec/macros/ - contains simulation macros and script files for Aldec ActiveHDL simulation tool
- create_design.do - ActiveHDL macro file, it creates all needed libraries and compiles all source files
- 08bit_tests - **TCL** script file. It invokes the ActiveHDL simulator for every test case in the /tests/d08* directories. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not.
- 16bit_tests - **TCL** script file. It invokes the ActiveHDL simulator for every test case in the /tests/d16* directories. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not.
- 32bit_tests - **TCL** script file. It invokes the ActiveHDL simulator for every test case in the /tests/d32* directories. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not.
- all_tests - **TCL** script file. It invokes the ActiveHDL simulator for every test case in /tests/d08*, /tests/d16* and /tests/d32* directories. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not.
- MAC_1G_MAC/tools/aldec/log/ - contains simulation reports files

1.4.6 Cadence NC-Sim support files

MAC_1G_AMBA/tools/cadence/nc_macros/ - directory contains simulation macros and script files for NC-SIM simulation tool.

- compile.do – NC-SIM macro file, it creates all needed libraries and compiles all source files
- simulate.do – NC_SIM macro file, it initializes the simulation with the test case placed in the *test/default* directory.
- wave.do – NC_SIM macro file, it creates a waveform with all vital signals from the MAC_1G_AMBA core and the Test Bench.
- readme.txt – A short description how to use all macro files.

MAC_1G_AMBA/tools/cadence/nc_scripts/ - contains a set of UNIX shell (C shell) scripts, DOS bat files and TCL script files for design compilation, and run for every test group and also for the whole test set

- compile – **UNIX** shell file. It creates all needed files (hdl.var, cds.lib), libraries and the compiles all needed source files.
- all_tests – **UNIX** shell file. . It invokes the NC_SIM simulator for whole test set. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not
- d8*_tests – **TCL** script files. It invokes the NC_SIM simulator for every test case in the /tests/d08* directories. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not.
- d16*_tests – **TCL** script files. It invokes the NC_SIM simulator for every test case in the /tests/d16* directories. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not.
- d32*_tests – **TCL** script files. It invokes the NC_SIM simulator for every test case in the /tests/d32* directories. As a result of this script work, there should be a log files in every of the test cases directory with the information if the test case pass or not.

MAC_1G_MAC/tools/cadence/lib/ – contains design libraries

MAC_1G_MAC/tools/ cadence /log/ – contains simulation reports files

1.4.7 Synopsys support files

MAC_1G_AMBA/tools/synopsys/dc_scripts/ - contains example scripts for Synopsys Design Compiler – synthesis tool,

compile.scr – Synopsys Design Compiler synthesis script for MAC_1G_AMBA core.

optimize_area.scr – Scripts for optimization for area

optimize_area_min.scr – Scripts for optimization for minimal area

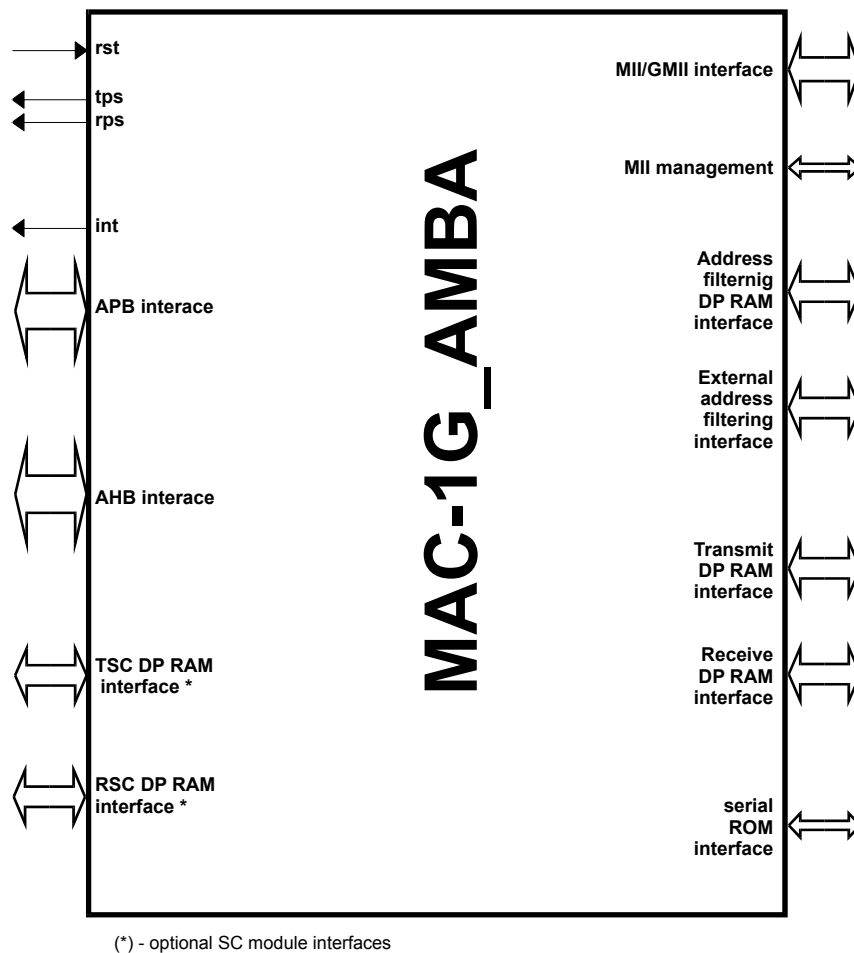
optimize_time.scr – Scripts for optimization for speed

optimize_time_max.scr – Scripts for optimization for maximal speed

readme.txt – Notes on how to run scripts

2 Hardware description

2.1 Symbol



2.2 Block diagram

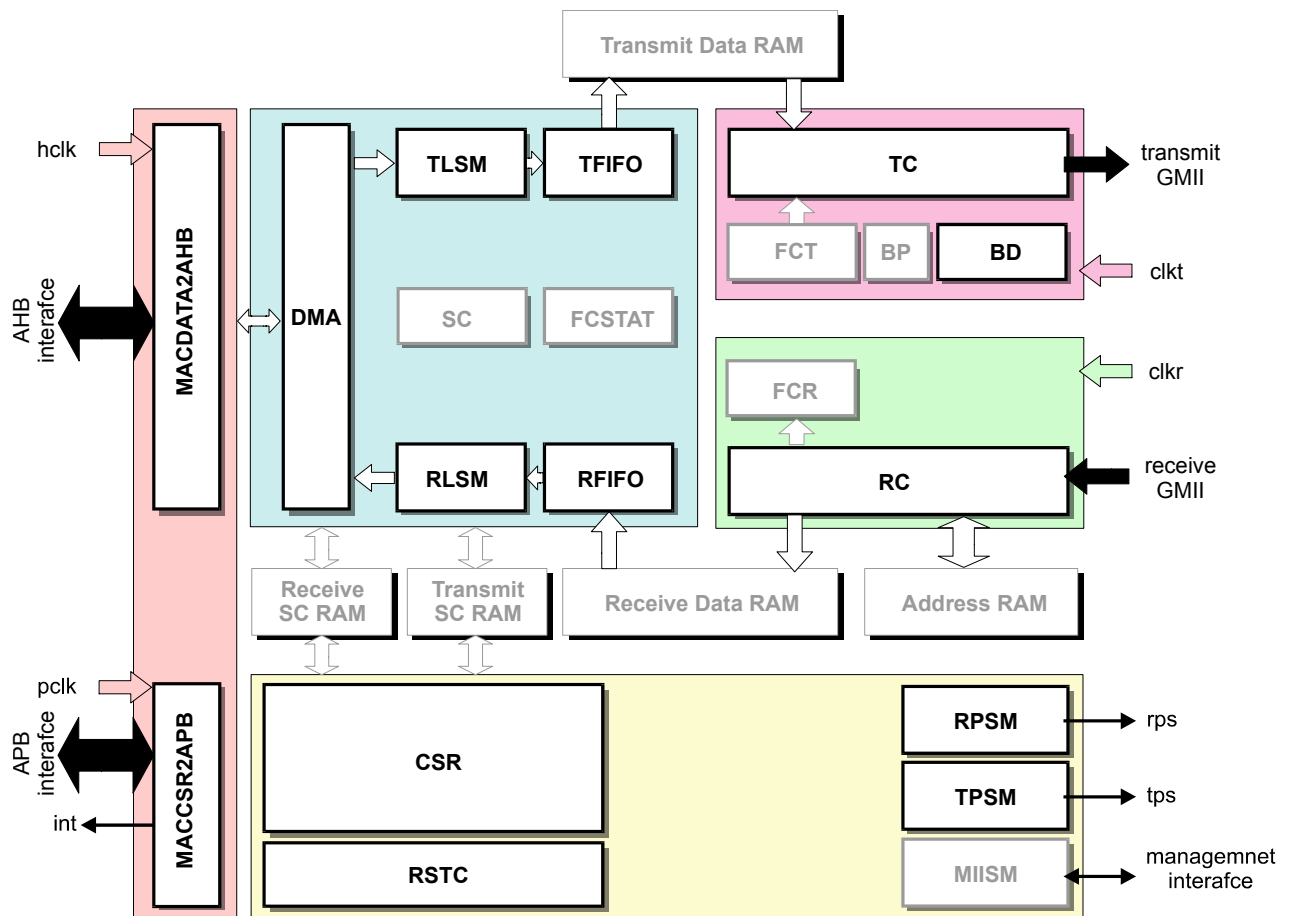


Figure 1. Block Diagram

2.2.1 TC – Transmit Controller

The transmit controller implements the 802.3 transmit operation. From the network side it uses the standard 802.3 1000Mb GMII/MII interface for an external PHY device. The transmit controller operates synchronously with the *clkt* clock from the GMII/MII interface

2.2.2 BD – Backoff/Deferring

The backoff/deferring controller implements the 802.3 half duplex operation. It operates synchronously with the *clkt* clock from the GMII/MII interface. The backoff/deferring controller can be optionally removed for lower gate count if the half duplex operation is not required.

2.2.3 RC – Receive Controller

The receive controller implements the 802.3 receive operation. From the network side it uses the standard 802.3 1000Mb GMII/MII interface for an external PHY device. The receive controller operates synchronously with the *clkr* clock from the GMII/MII interface.

2.2.4 TFIFO – Transmit FIFO

The transmit FIFO is used for buffering data prepared for transmission by the MAC-1G_AMBA. It provides an interface for the external dual-port RAM working as FIFO memory. The FIFO size can be configured by the generic parameters of the core. The TFIFODEPTH parameter defines the total FIFO size. The TCDEPTH parameter defines the maximum number of frames that can reside in the transmit FIFO at the moment. The transmit FIFO controller operates synchronously with the *clkdata* clock from the host Data interface.

2.2.5 RFIFO – Receive FIFO

The receive FIFO is used for buffering data received by the MAC-1G_AMBA. It provides an interface for the external dual-port RAM working as FIFO memory. The FIFO size can be configured by the generic parameters of the core. The RFIFODEPTH parameter defines the total FIFO size. The RCDEPTH parameter defines the maximum number of frames that can reside in the receive FIFO at the moment. The receive FIFO controller operates synchronously with the *clkdata* clock from the host Data interface.

2.2.6 TLSM – Transmit linked List State Machine

The transmit linked list state machine implements the descriptor/buffer architecture of the MAC-1G_AMBA. It manages the transmit descriptor list, and fetches the data prepared for transmission from the data buffers into the transmit FIFO. The transmit linked list state machine controller operates synchronously with the *clkdata* clock from the host Data interface.

2.2.7 RLSM – Receive linked List State Machine

The receive linked list state machine implements the descriptor/buffer architecture of the MAC-1G_AMBA. It manages the receive descriptor list, and moves the data the receive FIFO

into the data buffers. The receive linked list state machine controller operates synchronously with the *clkdata* clock from the host Data interface.

2.2.8 DMA – Direct Memory Access controller

The direct memory access controller implements the host Data interface. It services both the receive and the transmit channels. The direct memory access controller operates synchronously with the *clkdata* clock from the host Data interface.

2.2.9 MACDATA2AHB – AHB wrapper

The MAC-1G data to AHB interface wrapper. It translates MAC-1G data access operations to AHB control, address and data signals. The AHB wrapper component operates synchronously with the *hclk* clock from the host AHB interface

2.2.10 CSR – Control and Status Registers

The CSR component is used to control the MAC-1G_AMBA operation by the host. It implements the register set, the interrupt controller, and the power management functionality of the MAC-1G_AMBA. It also provides an interface for the host. The CSR component operates synchronously with the *clkcsr* clock from the host CSR interface.

2.2.11 MACCSR2APB – APB wrapper

The MAC-1G CSR to APB interface wrapper. It translate host read or write APB operations into MAC-1G CSR interface signals. The APB wrapper component operates synchronously with the *pclk* clock from the host APB interface.

2.2.12 RSTC – Reset Controller

The reset controller is used to reset all components of the MAC-1G_AMBA. It generates a reset signal synchronous to all clock domains in the design from the single external reset line.

2.2.13 MIISM – MII serial management

The MIISM interface controller is an optional module for MAC-1G_AMBA that provides a simple serial communication interface between MAC-1G_AMBA and the PHY(s). The module supply the hardware controlled protocol to read and/or write the status and configuration registers in PHY layer implementation.

2.2.14 SC – Statistical counters controller

The statistical counters controller is an optional module for MAC-1G_AMBA to capture MAC events. It relieves the host and software drivers of the responsibility of maintaining statistics for managed Ethernet port.

2.2.15 FCR – Flow control receiver

The flow control receiver module analyzes the incoming stream and capturing the Ethernet control frames. When the proper pause frame is detected the module delays a transmission of the next frames for a requested period of time.

2.2.16 FTC – Flow control transmitter

The flow control transmitter generates pause frames that are inserted into outgoing packet stream to instruct the party host to pause (or re-establish) a transmission. The pause frames are generated according to the actual receive FIFO usage level and a programmable threshold levels.

2.2.17 FCSTAT – Flow control statistics

The flow control statistics module generates signals that are used to increment a flow control oriented statistical counters

2.2.18 External components

There are three external components required for proper operation of the MAC-1G_AMBA core:

Receive data RAM	- synchronous dual port RAM working as receive FIFO.
Transmit data RAM	- synchronous dual port RAM working as transmit FIFO.
Receive SC RAM	- synchronous dual port RAM for the receive statistical counters
Transmit SC RAM	- synchronous dual port RAM for the transmit statistical counters
Address RAM	- synchronous dual port RAM working as addresses memory.

For more details concerning dual port RAMs refer to "External dual-port RAM interface" section of the MAC-1G_AMBA specification.

2.3 Pin description

Table 1. Pin description

Name	Type	Polarity/ Bus size	Description
General host interface signals			
rst	in	high	Host side reset
int	out	high	Interrupt
tps	out	high	Transmit process stopped
rps	out	high	Receive process stopped
gbo	out	high	Gigabit mode selection
APB slave interface			
pclk	in	rise	APB bus clock
presetn	in	low	APB bus reset
paddress	in	8	APB address bus
pselmaccsr	in	High	APB MAC CSR select
penable	in	High	APB transfer enable
pwrite	in	High	APB transfer direction
pwwdata	in	APBDATAWIDTH	APB write data bus
prdata	out	APBDATAWIDTH	APB read data bus
AHB master interface			
hclk	in	Rise	AHB bus clock
hresetn	in	Low	AHB bus reset
hrdata	in	AHBDATAWIDTH	AHB read data bus
hready	in	High	AHB transfer done
hresp	in	2	AHB slave transfer response
haddr	in	AHBADDRESSWIDTH	AHB address bus
htrans	out	2	AHB transfer type
hwrite	out	1	AHB transfer direction
hsize	out	3	AHB transfer size
hburst	out	3	AHB burst type
hprot	out	4	AHB protection control
hwdata	out	AHBDATAWIDTH	AHB write data bus
hgrantmac	in	High	AHB MAC bus grant
hbusreqmac	out	High	AHB MAB bus request
hlockmac	out	High	AHB MAC locked transfer
Transmit dual port RAM interface			
trdata	in	AHBDATAWIDTH	Transmit RAM read data
traddr	out	TFIFODEPTH	Transmit RAM read address
twdata	out	AHBDATAWIDTH	Transmit RAM write data
twe	out	High	Transmit RAM write enable
twaddr	out	TFIFODEPTH	Transmit RAM write address

Receive dual port RAM interface			
rrdata	in	AHBDATAWIDTH	Receive RAM read data
rraddr	out	RFIFODEPTH	Receive RAM read address
rdatao	out	AHBDATAWIDTH	Receive RAM data output
rwe	out	high	Receive RAM write enable
rwaddr	out	RFIFODEPTH	Receive RAM write address
Address filtering dual port RAM interface			
frdata	in	16	Address filtering RAM read data
fraddr	out	6	Address filtering RAM read address
fwe	out	high	Address filtering RAM write enable
fwaddr	out	6	Address filtering RAM write address
fwdata	out	16	Address filtering RAM write data
SC dual port RAM interface (clkcsr domain)			
sedr	in	32	Receive SC read value
scdt	in	32	Transmit SC read value
ssclrr	out	high	Single receive SC clear
ssclrt	out	high	Single transmit SC clear
scadr	out	4	Receive SC address
scadt	out	5	Transmit SC address
SC dual port RAM interface (clkdma domain)			
dir	in	32	Receive SC read data
dit	in	32	Transmit SC read data
ramwer	out	high	Receive SC RAM write enable
ramwet	out	high	Transmit SC RAM write enable
ar	out	4	Receive SC address
at	out	5	Transmit SC address
dor	out	32	Receive SC write data
Serial ROM memory interface			
sdi	in	1	Serial data
sclk	out	1	Serial clock output
sdo	out	1	Serial data output
External address filtering interface			
match	in	high	External address match
matchval	in	high	External address match valid
matchen	out	high	External address match enable
matchdata	out	48	Match data

GMII/MII PHY interface			
clkt	in	rise	Transmit clock
clkr	in	rise	Receive clock
rxer	in	high	GMII/MII Receive error
rxdv	in	high	GMII/MII Receive data valid
col	in	high	GMII/MII Collision detect
crs	in	high	GMII/MII Carrier sense
mdi	in	1	GMII/MII management data input
rxdata	in	8	GMII/MII Receive data
txen	out	high	GMII/MII Transmit enable
txer	out	high	GMII/MII Transmit error
mdc	out	rise	GMII/MII management clock
mdo	out	1	GMII/MII management data output
mden	out	high	GMII/MII management buffer control
txdata	out	8	GMII/MII Transmit data

2.4 Control and Status Registers (CSRs)

2.4.1 Control and Status Registers mapping

The Control and Status Registers are located physically on the MAC-1G_AMBA core and can be accessed directly by a host via a 8, 16 or 32-bit AMBA™ APB interface. All the CSRs are 32 bits long and quad word-aligned.

Table 2. Control and Status Registers locations

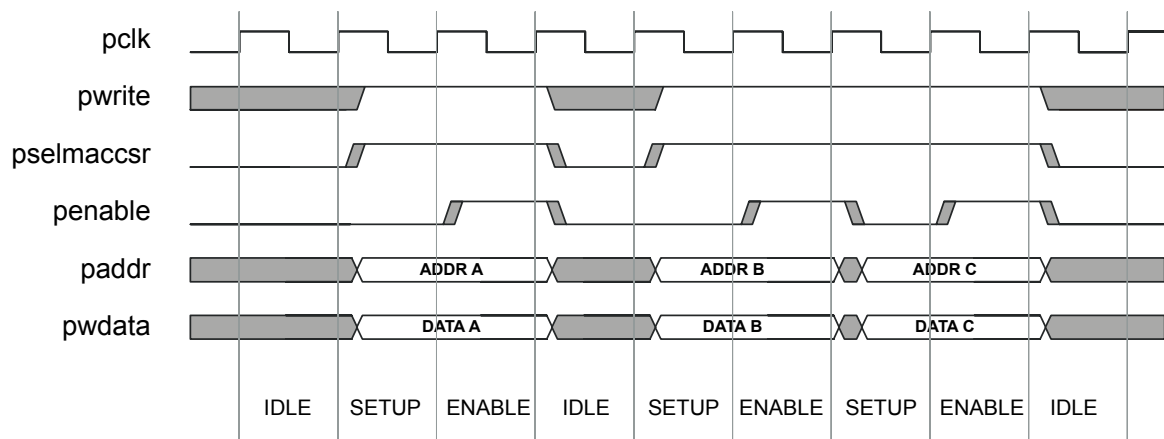
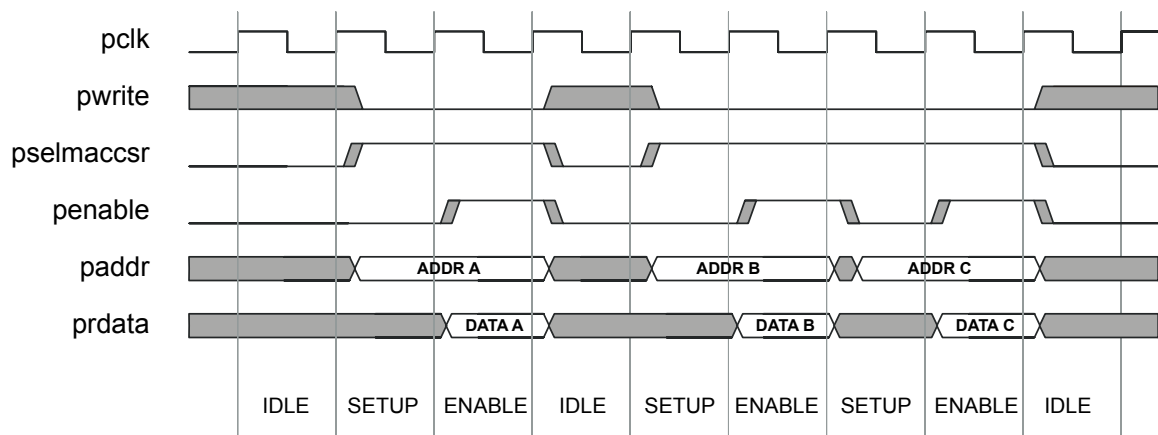
Register	Location	Reset value	Description
CSR0	00H	FE000000H	Bus mode
CSR1	08H	FFFFFFFFFH	Transmit poll demand
CSR2	10H	FFFFFFFFFH	Receive poll demand
CSR3	18H	FFFFFFFFFH	Receive list base address
CSR4	20H	FFFFFFFFFH	Transmit list base address
CSR5	28H	F0000000H	Status
CSR6	30H	32000040H	Operation mode
CSR7	38H	F3FE0000H	Interrupt enable
CSR8	40H	E0000000H	Missed frames and overflow counters
CSR9	48H	FFF483FFH	Serial ROM interface
CSR10	50H	30000000H	GMII/MII management
CSR11	58H	FFFE0000H	Timer and interrupt mitigation control
-	60H	-	not implemented
-	68H	-	not implemented
-	70H	-	not implemented
-	78H	-	not implemented
CSR16	0x80	0x00000000	MAC address low
CSR17	0x88	0x00000000	MAC address high
CSR18	0x90	0x00000000	Pause time & cache thresholds
CSR19	0x98	0x00000000	FIFO thresholds
CSR20	0xA0	0x03000000	Flow control setup & status
CSR21	0xA8	0x00000000	SC mode register (optional)
CSR22	0xB0	0x00000000	SC read value register (optional)

2.4.2 Control and Status Registers interface

The Control and Status Registers interface is a slave APB interface meaning that the host initiates write or read transactions. The APB interface operates synchronously with the *pclk* clock supplied by the system. The data width of the APB interface can be set with the generic "APBDATAWIDTH" parameter. Values permissible for the APBDATAWIDTH are 8, 16 and 32.

Table 3. Control and Status Register APB interface signals

Name	Type	Polarity/ Bus size	Description
pclk	in	Rise	APB interface clock. The rising edge of pclk is used to time all transfers on the APB.
presetn	in	Low	The APB bus reset signal is active LOW and this signal will normally be connected directly to the system bus reset signal.
pselmaccsr	in	High	Select CSR APB interface. This signal indicates that read and write operations on the APB bus are related to CSRs.
penable	in	High	This strobe signal is used to time all accesses on the bus. The enable signal is used to indicate the second cycle of an APB transfer. The rising edge of this signal occurs in the middle of the APB transfer.
pwrite	in	High	When HIGH this signal indicates an APB write access and when LOW a read access.
paddr	in	8	This is the APB address bus.
pwrdata	in	APBDATAWIDTH	The bus with a data, which are written to CSR when signal pwrite is high during APB data transfer.
prdata	out	APBDATAWIDTH	The bus with a data, which are read from CSR when signal pwrite is low during APB data transfer.

**Figure 2. APB write operations****Figure 3. APB read operations**

For detailed information about interface please refer to AMBA™ APB specification.

2.4.3 The bus mode register (CSR0)

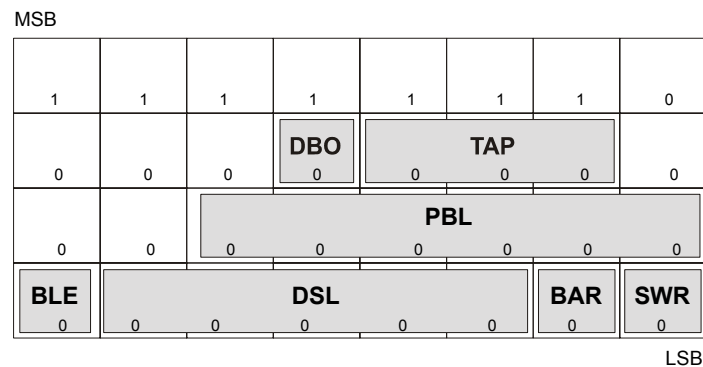


Figure 2. The bus mode register

Note:

The CSR0 register has unimplemented bits (shaded). If these bits are read they will return a predefined value. Writing these bits has no effect.

Table 4. The bus mode register bit functions

Bit	Symbol	Function
CSR0.20	DBO	Descriptor byte ordering mode. 1 – big endian mode used for data descriptors. 0 – little endian mode used for data descriptors.
CSR0.(19..17)	TAP	Transmit automatic polling. If the TAP is written with a nonzero value, the MAC-1G_AMBA performs an automatic transmit descriptor polling when operating in suspended state. When the descriptor is available, the transmit process goes into the running state. When the descriptor is marked as owned by the host, the transmit process remains suspended. The poll is always performed at the current transmit descriptor list position. The time interval between two consecutive polls is shown in Table 6.
CSR0.(13..8)	PBL	Programmable burst length. Specifies the maximum number of words that can be transferred within one DMA transaction. Values permissible are 0, 1, 2, 4, 8, 16, and 32. When a '0' value is written, the bursts are limited only by the internal FIFO's threshold levels. The width of the single word is equal to the CSRWIDTH generic parameter, i.e. all data transfers always use the maximum data bus width. Note that the PBL is valid only for the data buffers. The data descriptors burst length depends on a DATAWIDTH parameter. The rule is that every descriptor field (32-bit) is accessed with a single burst cycle. For DATAWIDTH=32, the descriptors are accessed with a single 32-bit word transaction, for DATAWIDTH=16 - a burst of two 16-bit words, and for DATAWIDTH=8 - a burst of four 8-bit words.

CSR0.7	BLE	Big/Little endian. Selects the byte-ordering mode used by the data buffers. 1 – big endian mode used for the data buffers. 0 – little endian mode used for the data buffers.
CSR0.(6..2)	DSL	Descriptor skip length. Specifies the number of longwords between two consecutive unchained descriptors.
CSR0.1	BAR	Bus arbitration scheme. 1 - transmit and receive processes have equal priority to access the bus. 0 - intelligent arbitration where the receive process has priority over the transmit process.
CSR0.0	SWR	Software reset. Setting this bit resets all internal flip-flops.

Table 5. Transmit automatic polling intervals

CSR0(19..17)	Time interval
000	TAP disabled
001	16.38us
010	49us
011	114.6us
100	1.024us
101	2.048us
110	3.072us
111	7.168us

2.4.4 The transmit poll demand register (CSR1)

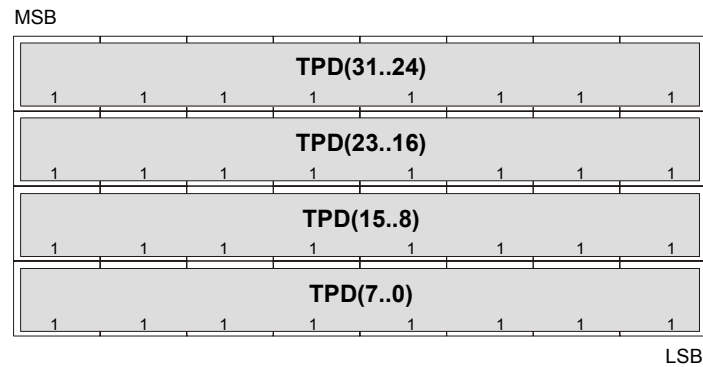


Figure 3. The transmit poll demand register

Table 6. The transmit poll demand bit functions

Bit	Symbol	Function
CSR1.(31..0)	TPD	Writing this field with any value instructs the MAC-1G_AMBA to check for frames to be transmitted. This operation is valid only when the transmit process is suspended. If no descriptor is available the transmit process remains suspended. When the descriptor is available the transmit process enters the running state.

2.4.5 The receive poll demand register (CSR2)

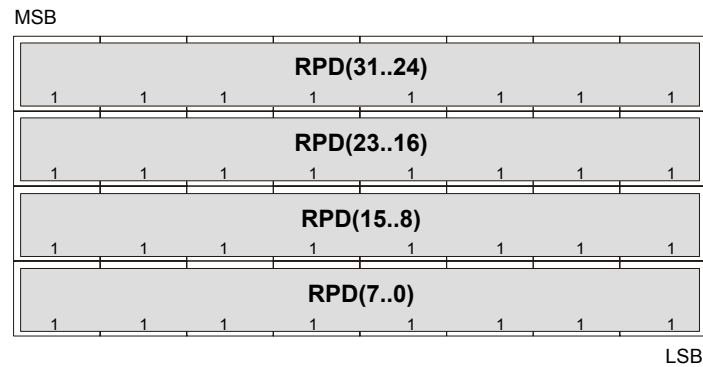


Figure 4. The receive poll demand register

Table 7. The receive poll demand bit functions

Bit	Symbol	Function
CSR2.(31..0)	RPD	Writing this field with any value instructs the MAC-1G_AMBA to check for receive descriptors to be acquired. This operation is valid only when the receive process is suspended. If no descriptor is available the receive process remains suspended. When the descriptor is available the receive process enters the running state.

2.4.6 The receive descriptor list base address register (CSR3)

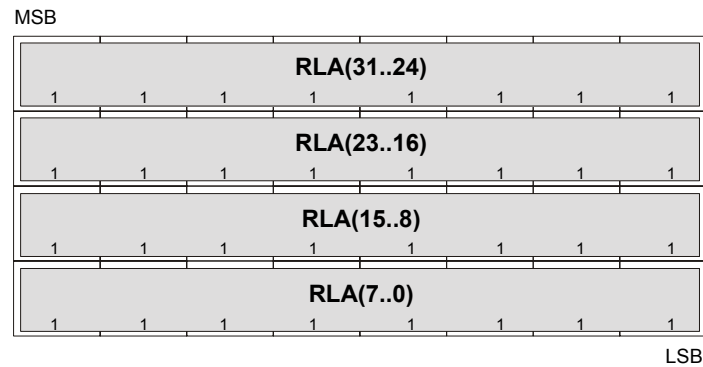


Figure 5. The receive descriptor list base address register

Table 8. The receive descriptor list base address register bit functions

Bit	Symbol	Function
CSR 3.(31..0)	RLA	Start of the receive list address. Contains the address of the first descriptor in a receive descriptor list. This address should be longword aligned (RLA(1..0)=00).

2.4.7 The transmit descriptor list base address register (CSR4)

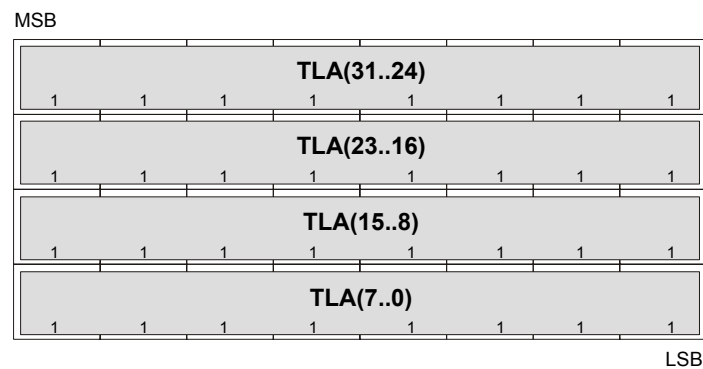


Figure 6. The transmit descriptor list base address register

Table 9. The transmit descriptor list base address register bit functions

Bit	Symbol	Function
CSR 4.(31..0)	TLA	Start of the transmit list address. Contains the address of the first descriptor in a transmit descriptor list. This address should be longword aligned (TLA(1..0)=00).

2.4.8 The status register (CSR5)

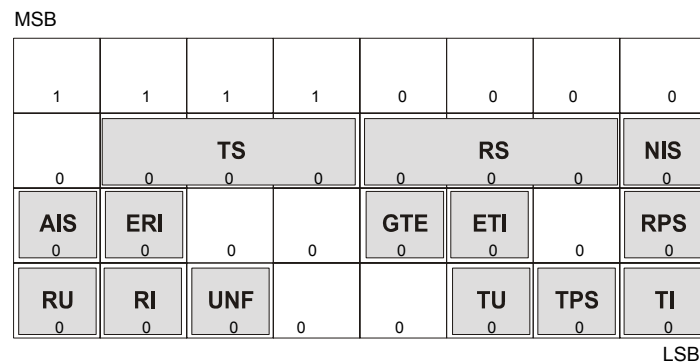


Figure 7. The status register

Note:

The CSR5 register has unimplemented bits (shaded). If these bits are read they will return a predefined value. Writing these bits has no effect.

Table 10. The status register bit functions

Bit	Symbol	Function
CSR5.(22..20)	TS	Transmit process state (Read only). Indicates the current state of a transmit process: 000 - Stopped, RESET or STOP TRANSMIT command issued 001 - Running, fetching transmit descriptor. 010 - Running, waiting for end of transmission. 011 - Running, transferring data buffer from host memory to FIFO. 100 - Reserved. 101 - Running, setup packet. 110 - Suspended, FIFO underflow or unavailable descriptor. 111 - Running, closing transmit descriptor.
CSR5.(19..17)	RS	Receive process state (Read only). Indicates the current state of a receive process: 000 - Stopped, RESET or STOP RECEIVE command issued. 001 - Running, fetching receive descriptor. 010 - Running, waiting for the end of receive packet before prefetch of next descriptor. 011 - Running, waiting for receive packet. 100 - Suspended, unavailable receive buffer. 101 - Running, closing receive descriptor. 110 - Not used 111 - Running, transferring data from FIFO to host memory.

Bit	Symbol	Function
CSR5.16	NIS	Normal interrupt summary. This bit is logical or on the following bits: CSR5.0 - Transmit interrupt CSR5.2 - Transmit buffer unavailable CSR5.6 - Receive interrupt CSR5.11 - General purpose timer overflow CSR5.14 - Early receive interrupt Only the unmasked bits affect the normal interrupt summary bit. The user can clear this bit by writing a '1'. Writing a '0' has no effect.
CSR5.15	AIS	Abnormal interrupt summary. This bit is logical or on the following bits: CSR5.1 - Transmit process stopped CSR5.5 - Transmit underflow CSR5.7 - Receive buffer unavailable CSR5.8 - Receive process stopped CSR5.10 - Early transmit interrupt Only the unmasked bits affect the abnormal interrupt summary bit. The user can clear this bit by writing a '1'. Writing a '0' has no effect.
CSR5.14	ERI	Early receive interrupt. Set when the MAC-1G_AMBA fills the data buffers of the first descriptor. Cleared by the MAC-1G_AMBA after the receive interrupt (CSR5.6). The user can clear this bit by writing 1. Writing 0 has no effect.
CSR5.11	GTE	General-purpose timer expiration. Set when the general-purpose timer reaches a value of zero. The user can clear this bit by writing a '1'. Writing a '0' has no effect.
CSR5.10	ETI	Early transmit interrupt. Indicates that the packet to be transmitted was fully transferred into the FIFO. This bit is cleared by the MAC-1G_AMBA after the transmit interrupt (CSR5.0). The user can clear this bit by writing a '1'. Writing a '0' has no effect.
CSR5.8	RPS	Receive process stopped. RPS is set when a receive process enters a stopped state. The user can clear this bit by writing a '1'. Writing a '0' has no effect.

Bit	Symbol	Function
CSR5.7	RU	Receive buffer unavailable. When set, indicates that the next receive descriptor is owned by the host and is unavailable for the MAC-1G_AMBA. When RU becomes set, the MAC-1G_AMBA enters a suspended state, and returns to receive descriptor processing when the host changes ownership of the descriptor and either a receive poll demand command is issued or a new frame is recognized by the MAC-1G_AMBA. The user can clear this bit by writing a '1'. Writing a '0' has no effect.
CSR5.6	RI	Receive interrupt. Indicates the end of a frame receive. The complete frame has been transferred into the receive buffers. The user can clear this bit by writing a '1'. Writing a '0' has no effect.
CSR5.5	UNF	Transmit underflow. Indicates that the transmit FIFO was empty during a transmission. The transmit process goes into a suspended state. The user can clear this bit by writing a '1'. Writing a '0' has no effect.
CSR5.2	TU	Transmit buffer unavailable. When set, TU indicates that the host owns the next descriptor on the transmit descriptor list therefore it cannot be used by the MAC-1G_AMBA. When the TU becomes set, the transmit process goes into a suspended state and can resume normal descriptor processing when the host changes ownership of the descriptor and either a transmit poll demand command is issued or transmit automatic polling is enabled. The user can clear this bit by writing a '1'. Writing a '0' has no effect.
CSR5.1	TPS	Transmit process stopped. TPS is set when the transmit process goes into a stopped state. The user can clear this bit by writing a '1'. Writing a '0' has no effect.
CSR5.0	TI	Transmit interrupt. Indicates the end of a frame transmission process. The user can clear this bit by writing a '1'. Writing a '0' has no effect.

2.4.9 The operation mode register (CSR6)

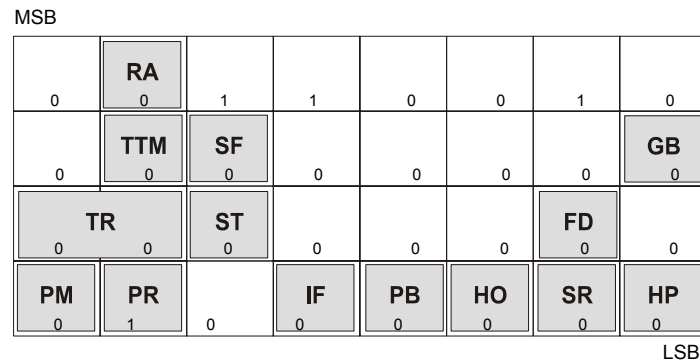


Figure 8. The operation mode register

Note:

The CSR6 register has unimplemented bits (shaded). If these bits are read they will return a predefined value. Writing these bits has no effect.

Table 11. The operation mode register bit functions

Bit	Symbol	Function
CSR6.30	RA	Receive all. When set, all incoming frames are received regardless of their destination address. An address check is performed and the result of the check is written into the receive descriptor (RDES0.30).
CSR6.22	TTM	Transmit threshold mode. This bit can be changed only when a transmit process is in a stopped state.
CSR6.21	SF	Store and forward. When set, the transmission starts after a full packet is written into the transmit FIFO, regardless of the current FIFO threshold level. This bit can be changed only when the transmit process is in the stopped state.
CSR6.16	GB	Gigabit Selection. GB = 1 selects 1Gbps mode GB = 0 selects 10/100Mbps mode This bit can be changed only when both the transmit and the receive processes are in the stopped state.
CSR6.(15..14)	TR	Threshold control bits. This bit, together with the TTM, the SF and the PS, control the threshold level for the transmit FIFO.

Bit	Symbol	Function
CSR6.13	ST	Start/stop transmit command. Setting this bit when the transmit process is in a stopped state causes transition into a running state. In the running state the MAC-1G_AMBA checks the transmit descriptor at a current descriptor list position. If the MAC-1G_AMBA owns the descriptor then the data starts to transfer from memory into the internal transmit FIFO. When the host owns the descriptor, the MAC-1G_AMBA enters a suspended state state. Clearing this bit when the transmit process is in a running or a suspended state instructs the MAC-1G_AMBA to enter the stopped state.
CSR6.9	FD	Full duplex mode. 0 – half duplex mode. 1 – forcing full duplex mode. Changing of this bit is allowed only when both the transmitter and receiver processes are in the stopped state.
CSR6.7	PM	Pass all multicast. When set, all the frames with the multicast destination addresses will be received regardless of the address check result.
CSR6.6	PR	Promiscuous mode. When set all the frames will be received regardless of the address check result. An address check is not performed.
CSR6.4	IF	Inverse filtering (Read only). If this bit is set when working in a perfect filtering mode, the receiver performs an inverse filtering during the address check process. The "filtering type" bits of the setup frame determine a state of this bit.
CSR6.3	PB	Pass bad frames. When set, the MAC-1G_AMBA transfers all frames into the data buffers, regardless of the receive errors. This allows the runt frames, collided fragments and truncated frames to be received.
CSR6.2	HO	Hash only filtering mode (Read only). When set, the MAC-1G_AMBA performs an imperfect filtering over both the multicast and the physical addresses. The "filtering type" bits of the setup frame determine the state of this bit.

Bit	Symbol	Function
CSR6.1	SR	Start/stop receive command. Setting this bit when the receive process is in a stopped state causes the transition into a running state. In the running state the MAC-1G_AMBA checks the receive descriptor at the current descriptor list position. If the MAC-1G_AMBA owns the descriptor, then it can process an incoming frame. When the host owns the descriptor, the receiver enters a suspended state and also sets the CSR5.7 (receive buffer unavailable) bit. Clearing this bit when the receive process is in running or suspended state instructs the MAC-1G_AMBA to enter a stopped state after receiving the current frame.
CSR6.0	HP	Hash/perfect receive filtering mode (Read only). 0 – perfect filtering of the incoming frames is performed according to the physical addresses specified in a setup frame. 1 – imperfect filtering over the frames with the multicast addresses is performed according to the hash table specified in a setup frame. A physical addresses check is performed according to the CSR6.2 (HO - Hash only) bit. When the HO and HP are both set, an imperfect filtering is performed on all of the addresses. The "filtering type" bits of the setup frame determine the state of this bit.

Table 13 lists all the possible combinations of the address filtering bits. The actual values of the IF, HO, and HP bits are determined by the filtering type (FT1:FT0) bits in the setup frame. The IF, HO, and HP bits are read-only bits.

Table 12. Receive address filtering modes summary

PM CSR6.7	PR CSR6.6	IF CSR6.4	HO CSR6.2	HP CSR6.0	Current Filtering mode
0	0	0	0	0	16 physical addresses – perfect filtering mode
0	0	0	0	1	1 physical address for physical addresses and 512-bit hash table for multicast addresses
0	0	0	1	1	512-bit hash table for both physical and multicast addresses
0	0	1	0	0	Inverse filtering
x	1	0	0	x	Promiscuous mode
0	1	0	1	1	Promiscuous mode
1	0	0	0	x	Pass all multicast frames
1	0	0	1	1	Pass all multicast frames

Table 14 lists the transmit FIFO threshold levels. These levels are specified in bytes. Because of a FIFOs' configurability, some of these levels can be invalid for some FIFO configurations.

Table 13. Transmit FIFO threshold levels (bytes)

CSR6.21	CSR6.15..14	CSR6.22=1	CSR6.22=0
0	00	64	128
0	01	128	256
0	10	128	512
0	11	256	1024
1	xx	Store and Forward	Store and Forward

2.4.10 The interrupt enable register (CSR7)

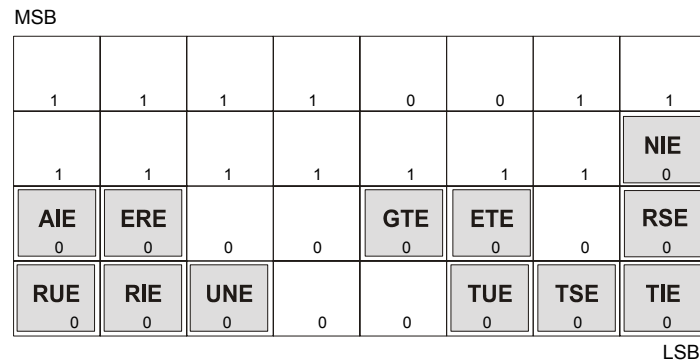


Figure 9. The interrupt enable register

Note:

The CSR7 register has unimplemented bits (shaded). If these bits are read they will return a predefined value. Writing these bits has no effect.

Table 14. The interrupt enable register bit functions

Bit	Symbol	Function
CSR7.16	NIE	Normal interrupt summary enable. When set, normal interrupts are enabled. Normal interrupts are listed below: CSR5.0 – Transmit interrupt CSR5.2 – Transmit buffer unavailable CSR5.6 – Receive interrupt CSR5.11 – General-purpose timer expired CSR5.14 – Early receive interrupt
CSR7.15	AIE	Abnormal interrupt summary enable. When set, abnormal interrupts are enabled. Abnormal interrupts are listed below: CSR5.1 – Transmit process stopped CSR5.5 – Transmit underflow CSR5.7 – Receive buffer unavailable CSR5.8 – Receive process stopped CSR5.10 – Early transmit interrupt
CSR7.14	ERE	Early receive interrupt enable. When both the ERE and normal interrupt enable bits are set, early receive interrupt is enabled.
CSR7.11	GTE	General-purpose timer overflow enable. When both the GTE and normal interrupt summary enable bits are set, the general-purpose timer overflow interrupt is enabled.
CSR7.10	ETE	Early transmit interrupt enable. When both the ETE and abnormal interrupt summary enable bits are set, the early transmit interrupt is enabled.
CSR7.8	RSE	Receive stopped enable. When both the RSE and abnormal interrupt summary enable bits are set, the receive stopped interrupt is enabled.

Bit	Symbol	Function
CSR7.7	RUE	Receive buffer unavailable enable. When both the RUE and abnormal interrupt summary enable bits are set, the receive buffer unavailable is enabled.
CSR7.6	RIE	Receive interrupt enable. When both the RIE and normal interrupt summary enable bits are set, the receive interrupt is enabled.
CSR7.5	UNE	Underflow interrupt enable. When both the UNE and abnormal interrupt summary enable bits are set, the transmit underflow interrupt is enabled.
CSR7.2	TUE	Transmit buffer unavailable enable. When both the TUE and normal interrupt summary enable bits are set, the transmit buffer unavailable interrupt is enabled.
CSR7.1	TSE	Transmit stopped enable. When both the TSE and abnormal interrupt summary enable bits are set, the transmit process stopped interrupt is enabled.
CSR7.0	TIE	Transmit interrupt enable. When both the TIE and normal interrupt summary enable bits are set, the transmit interrupt is enabled.

2.4.11 The missed frames and overflow counter register (CSR8)

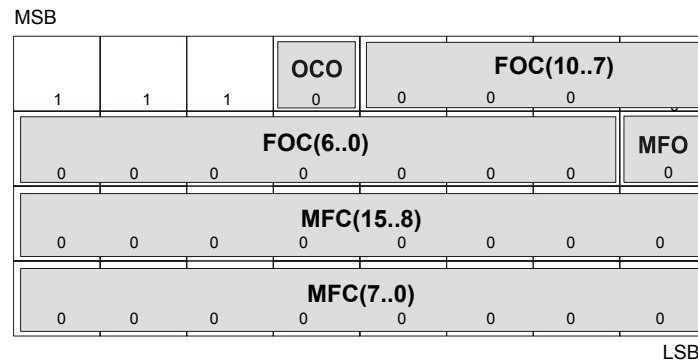


Figure 10. The missed frames and overflow counter register

Note:

The CSR8 register has unimplemented bits (shaded). If these bits are read they will return a predefined value. Writing these bits has no effect.

Table 15. The missed frames and overflow counter bit functions

Bit	Symbol	Function
CSR8.28	OCO	Overflow counter overflow (read only). Gets set when the FIFO overflow counter overflows. Reset when read.
CSR8.(27..17)	FOC	FIFO overflow counter (read only). Counts the number of frames not accepted due to the receive FIFO overflow. The counter resets when read.
CSR8.16	MFO	Missed frame overflow. Set when a missed frame counter overflows. Reset when read.
CSR8.(15..0)	MFC	Missed frame counter (read only). Counts the number of frames not accepted due to the unavailability of the receive descriptor. The counter resets when read.

2.4.12 The serial ROM register (CSR9)

MSB							
1	1	1	1	1	1	1	1
1	1	1	1	0	1	0	0
1	0	0	0	0	0	1	1
				SDO	SDI	SCLK	SCS
1	1	1	1	1	1	1	1
LSB							

Figure 11. The serial ROM register

Note:

The CSR9 register has unimplemented bits (shaded). If these bits are read they will return a predefined value. Writing these bits has no effect.

The Serial ROM interface can be used to access external Serial ROM device via the CSR registers of the MAC. The user can supply an external Serial ROM device as shown on Figure 14. The serial ROM can be used for storing the user data, such as ethernet addresses. Note that all access sequences and timing of the Serial ROM interface are handled by the software.

If the serial ROM interface is not used, the input pin should be stuck at '0' and all output pins should be left floating.

Table 16. The serial ROM register bit functions

Bit	Symbol	Function
CSR9.3	SDO	Serial ROM data output. The value of this bit drives the <i>sdo</i> pin of the MAC.
CSR9.2	SDI	Serial ROM data input. This bit reflects the <i>sdi</i> pin of the MAC.
CSR9.1	SCLK	Serial ROM clock. The value of this bit drives the <i>sclk</i> pin of the MAC.
CSR9.0	SCS	Serial ROM chip select. The value of this bit drives the <i>scs</i> pin of the MAC.

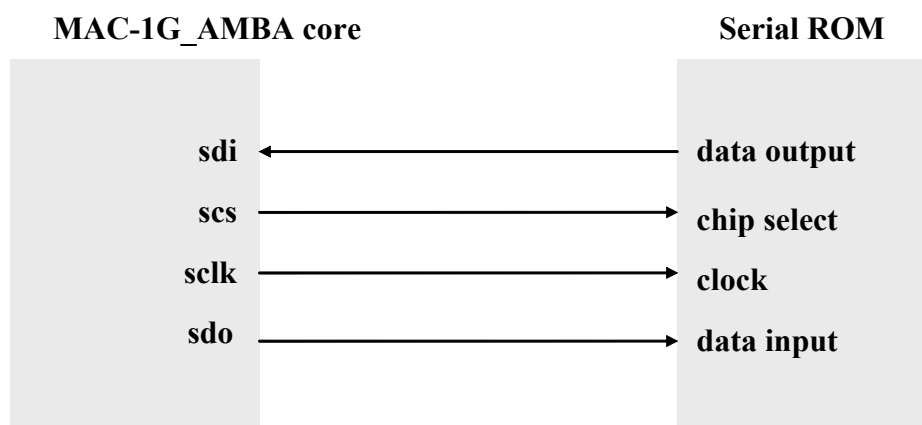


Figure 12. External Serial ROM connections

2.4.13 GMII/MII Management register (CSR10)

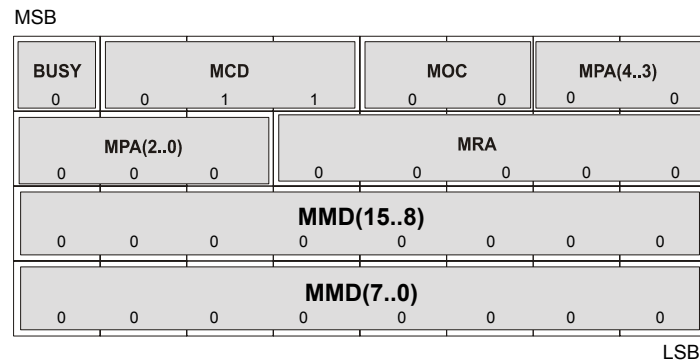


Figure 13. The GMII/MII management register

MII Management Interface is used to configure and control physical layer over MII interface. Information transferred between MAC and PHY over interface consists of two signals MDC and MDIO. MDC (Management Data Clock) is sourced by the Station Management entity (MAC) to the PHY as timing reference for transfer of information on the MDIO signal. MDC has a maximum clock rate at 2.5 MHz (according to IEEE 802.3 specification) and no minimum rate. MDIO (Management Data Input/Output) is a bi-directional signal used to transfer control information and status between PHY and MAC and may be shared by up to 32 devices.

Management data are transferred in frames. Every frame has a structure consist of preamble, operation code, PHY and register addresses and transferred data. The PHY address allows select appropriate transceiver from the connected to the management interface. The register address selects one from thirty-two 16-bit control and status registers which can be read or write, respectively to used operation code.

The MDIO pin requires a pull-up resistor which, during IDLE and turnaround, will pull MDIO high. In order to initialize the MDIO interface, the MIISM interface controller at station management entity sends a sequence of 32 continuous logic ones (preamble) on MDIO (or drives MDIO in high impedance state to let the pull-up resistor stimulate the logic one on signal for 32 clock cycles). This process provides the PHY entity with a sequence that is used to establish synchronization. The PHY waits until it has received such preamble sequence before responding to any transaction

End of preamble and start transfer process is indicated by Start Code (a "01" pattern). This assures the MDIO line transitions from the default idle line state. Then station management entity drives MDIO line with a bits sequence of Operation Code, PHY Address and Register Address fields. Turnaround is defined as an idle bit time inserted between the Register Address field and the Data field. To avoid contention during a read transaction, no device shall actively drive the MDIO signal during the first bit of Turnaround. After turnaround data bits are transferred.

Table 17. MIISM frame format

MIISM Operation	<idle> [<pre>] <start> <opcode> <device addr> <reg addr> <turnaround> <data> <idle>
Read operation	<idle> [<pre>] <01> <10> <AAAAA> <RRRRR> <Z0> <DDDD DDDD DDDD DDDD>
Write operation	<idle> [<pre>] <01> <01> <AAAAA> <RRRRR> <10> <DDDD DDDD DDDD DDDD>

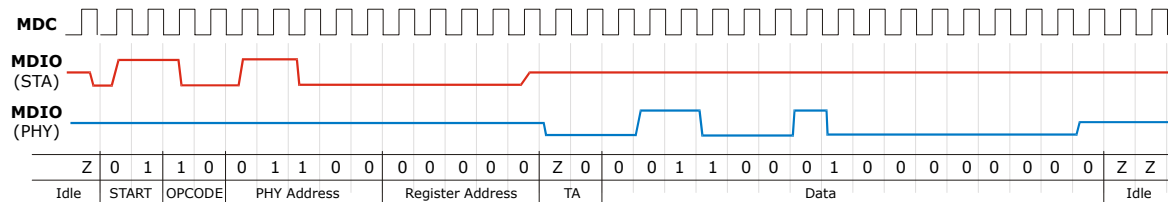


Figure 14. Typical MII serial management read operation

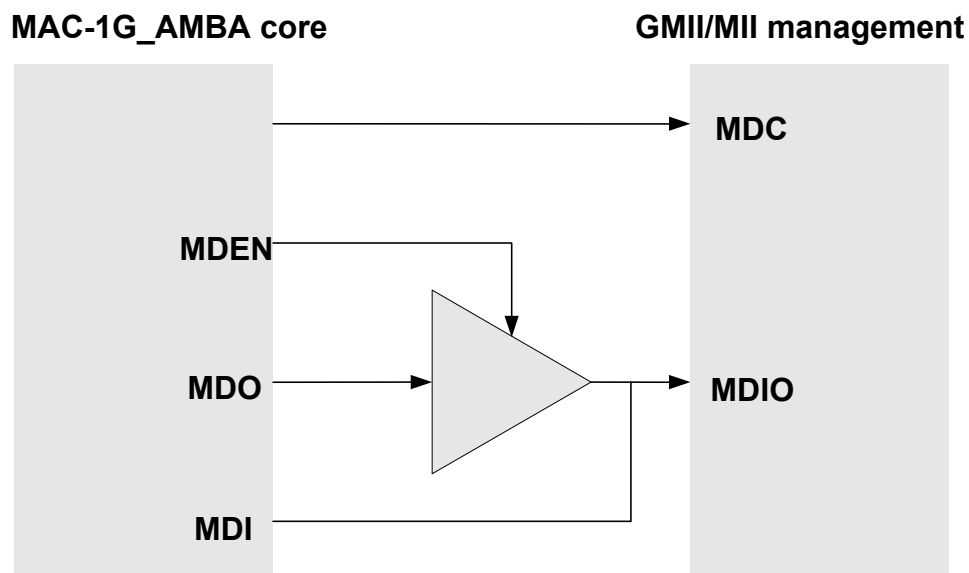


Figure 15. External tristate buffer connections

Table 18. GMII/MII Management register bit functions

Bit	Symbol	Function
CSR10.31	START/ BUSY	Writing this bit with "1" starts management data transfer or applying MIISM settings respectively to used opcode. When read as "1" indicates that data transition / setting applying process is in progress; When read as "0" indicates data transition / setting applying process completion.
CSR10.(30..28)	CLKDIV	When register is read this field contains actual MIISM clock divider settings Writing register with new CLKDIV value simultaneously with OPCODE set to "11" and START bit set applying new clock divider settings on next rising edge of MDCLK signal. Clock divider settings: "000" - MDC = clk / 8 "001" - MDC = clk / 16 "010" - MDC = clk / 32 "011" - MDC = clk / 64 "100" - MDC = clk / 128 "101" - MDC = clk / 256 "110" - MDC = clk / 512 "111" - MDC = clk / 1024 The default value is "011".
CSR10.(27..26)	OPCODE*	MIISM operation code: "01" - register read command, "10" - register write command, "11" - clock divider set.
CSR10.(25..21)	PHYADD*	Physical layer address for current transfer
CSR10.(20..16)	REGADD*	Register address for the current transfer.
CSR10.(15..0)	DATA*	Register data. When write operation is selected (OPCODE="10"), the data from this field will be written to selected register. When register read code is used (OPCODE="01"), the field will be overwritten by the data from selected register.

* **Note:** register bits 27..0 are identical to last 28 bits of MII management frame when omit two turnaround bits.

2.4.14 The general-purpose timer and interrupt mitigation control register (CSR11)

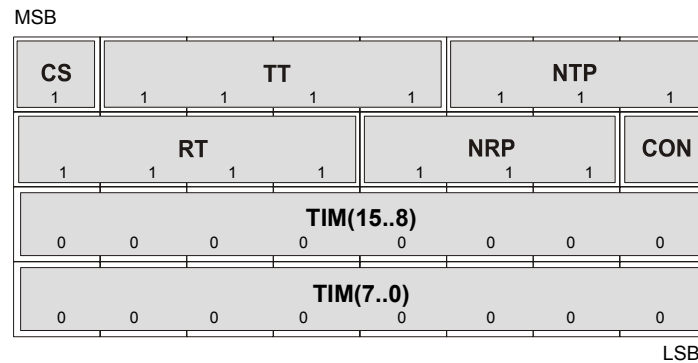


Figure 16. The general-purpose timer and interrupt mitigation control register

Table 19. The general-purpose timer and interrupt mitigation control bit functions

Bit	Symbol	Function
CSR11.31	CS	Cycle size. Controls the time units for the transmit and receive timers according to the following table: 1 – 1.024us, 0 – 16.384us,
CSR11.(30..27)	TT	Transmit timer. Controls the time that must elapse between the end of a transmit operation and issuing the transmit interrupt. This time is equal to TT * (16*CS). Writing '0' to this field disables the timer effect on the transmit interrupt mitigation mechanism.
CSR11.(26..24)	NTP	Number of transmit packets. Controls the number of the frames transmitted before issuing the transmit interrupt. Writing '0' to this field disables the counter effect on the transmit interrupt mitigation mechanism.
CSR11.(23..20)	RT	Receive timer. Controls the time that must elapse between the end of a receive operation and issuing the receive interrupt. This time is equal to RT * CS. Writing '0' to this field disables the timer effect on the receive interrupt mitigation mechanism.
CSR11.(19..17)	NRP	Number of receive packets. Controls the number of the received frames before issuing the receive interrupt. Writing '0' to this field disables the timer effect on the receive interrupt mitigation mechanism.
CSR11.16	CON	Continuous mode. 1 – general-purpose timer works in continuous mode 0 – general-purpose timer works in one-shot mode
CSR11.(15..0)	TIM	Timer value. Contains the number of iterations of the general-purpose timer. Each iteration duration is 16.384us

2.4.15 MAC address low register (CSR16)

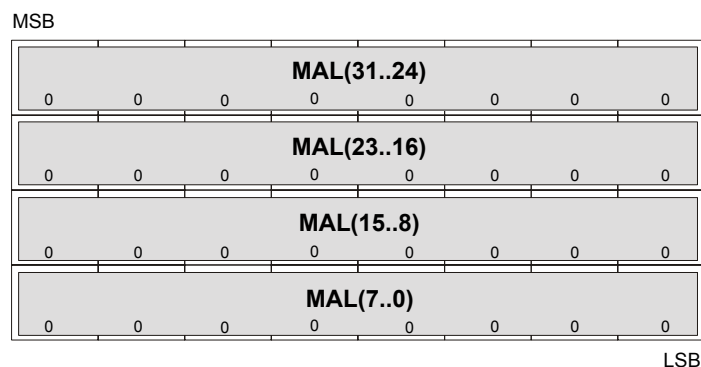


Figure 17. MAC address low register

Table 20. The MAC address low register bit functions

Bit	Symbol	Function
CSR16.(31..0)	MAL	MAC address low bytes. This field contains 4 lowest bytes of the MAC hardware address (MAL(31..0) = MAC(31..0)). This field is used to fill the source hardware address field of pause frames generated by host.

2.4.16 MAC address high register (CSR17)

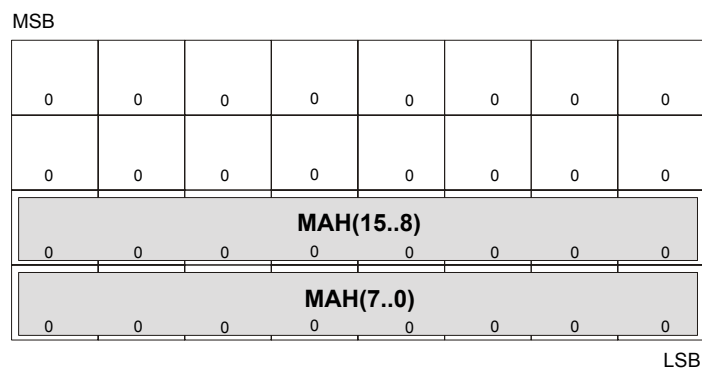


Figure 18. MAC address high register

Table 21. The MAC address low register bit functions

Bit	Symbol	Function
CSR17.(15..0)	MAH	MAC address high bytes. This field contains 2 highest bytes of the MAC hardware address (MAH(15..0) = MAC(63..32)). This field is used to fill the source hardware address field of pause frames generated by host.

2.4.17 Pause time & cache thresholds register (CSR18)

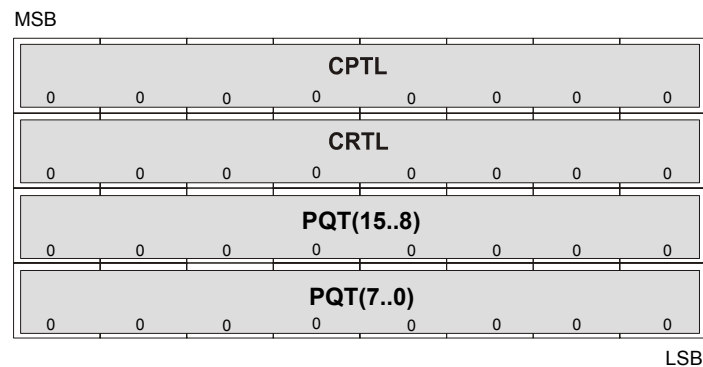


Figure 19. Pause time & cache thresholds register

Table 22. Pause time & cache thresholds register bit functions

Bit	Symbol	Function
CSR18.(31..24)	CPTL	Cache pause threshold level. The field contains unsigned value indicating the minimum number of frames stored in the receive FIFO when the flow control mechanism starts to transmit pause frames. The actual width of the field depends of RCDEPTH parameter and a maximum value is 2RCDEPTH-1 (the significant bits are from 24 to 23+RCDEPTH). Writing a value less than in a CRTL field or 0 disables the flow control reactions on frame cache overflow..
CSR18.(23..16)	CRTL	Cache restart threshold level. The field contains unsigned value indicating the maximum number of frames stored in the receive FIFO below which the flow control mechanism stops to transmit pause frames. The actual width of the field depends of RCDEPTH parameter and a maximum value is 2RCDEPTH-1 (the significant bits are from 16 to 15+RCDEPTH). Writing a value larger than in a CPTL field or 0 disables the flow control reactions on frame cache overflow.
CSR18.(15..0)	PQT	Flow control pause quanta time. This field is used to fill the pause_time field of outgoing flow-control pause frames. The PQT contains 2-octet long unsigned integer indicating the length of time for which the receiving station will be requested to inhibit next data frame transmission The pause_time is measured in units of pause_quanta, equal to 512 bit times of the particular implementation (10/100/1000 Mbit/s) The range of possible pause_time is 0 to 65535 pause_quanta.* Writing a value of 0 disables the flow control protocol.

2.4.18 FIFO thresholds register (CSR19)

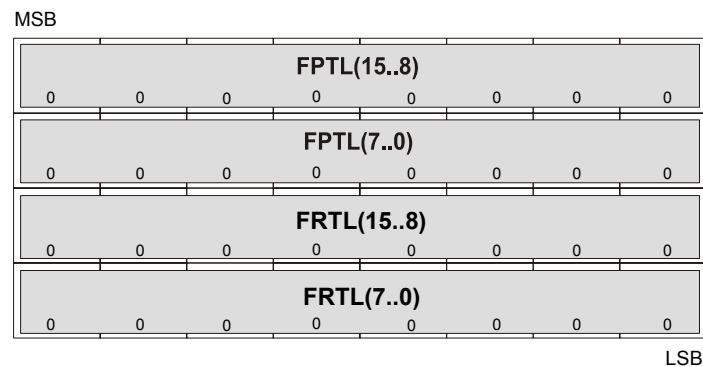


Figure 20. Pause time & cache thresholds register

Table 23. Pause time & cache thresholds register bit functions

Bit	Symbol	Function
CSR19.(31..16)	FPTL	FIFO pause threshold level. The field contains unsigned value indicating the minimum number of bytes stored in the receive FIFO when the flow control mechanism starts to transmit pause frames. The actual width of the field depends of RFIFODEPTH parameter and a maximum value is $2 \times \text{RFIFODEPTH} - 1$ (the significant bits are from 16 to $15 + \text{RFIFODEPTH}$). Writing a value less than in a FRTL field or 0 disables the flow control reactions on FIFO overflow.
CSR19.(15..0)	FRTL	FIFO restart threshold level. The field contains unsigned value indicating the maximum number of bytes stored in the receive FIFO below which the flow control mechanism stops to transmit pause frames. The actual width of the field depends of RFIFODEPTH parameter and a maximum value is $2 \times \text{RFIFODEPTH} - 1$ (the significant bits are from 0 to $\text{RFIFODEPTH} - 1$). Writing a value larger than in a FPTL field or 0 disables the flow control reactions on frame cache overflow.

2.4.19 Flow control setup & status (CSR20)

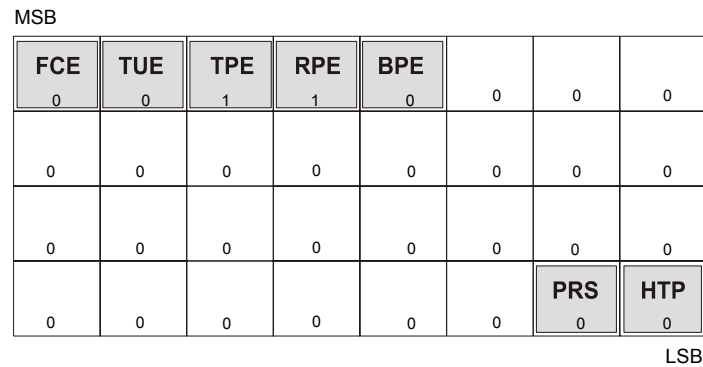


Figure 21. Flow control setup & status

Table 24. Flow control setup & status bit functions

Bit	Symbol	Function
CSR20.31	FCE	Flow Control Enable Writing this field bit 1 enables all full and half-duplex flow control modules. The functionality of full-duplex flow control can be configured with TUE, TPE and RPE bits. The functionality of half-duplex flow control (back pressure) can be configured with BPE bit.
CSR20.30	TUE	Transmit Un-pause frames Enable Writing this bit with 1 enables the transmission of pause frames with 0 pause time when the FIFO is near emptied (below the programmable threshold levels).
CSR20.29	TPE	Transmit Pause frames Enable Writing this field with 1 enables the transmission of pause frames with when the FIFO is near overflow (above the programmable threshold levels).
CSR20.28	RPE	Receive Pause frames Enable. Setting this bit with 1 enables the reception of pause frames and delaying the transmissions.
CSR20.1	PRS	Pause Request Send. This bit is read only. When read as “1” indicates that the host has sent a pause frame and the receiver should be delaying the transmission of next frame now; otherwise read as “0”.
CSR20.0	HTP	Host Transmission Paused. This bit is read only. When read as “1” indicates that the host is now delaying the transmission of the next frame according to the received pause frame; otherwise read as “0”.

2.4.20 The SC mode register (CSR21)

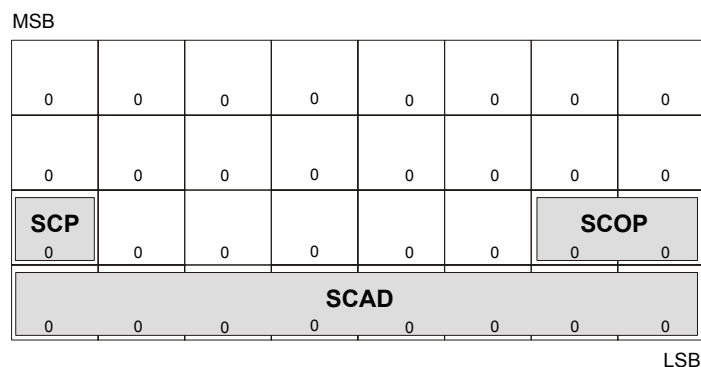


Figure 22. SC mode register

Table 25. SC mode register bit functions

Bit	Symbol	Function
CSR21.15	SCP	Setting this bit allows reading and clearing single SC counter. Writing '1' to this bit, causes that user can not perform any operation on SC counters, until this bit is not set. When it is set, any operation on SC can be performed. After realization all needed operation on SC, this bit should be cleared. Clearing this bit is realized by writing '0' to this bit and no operation on SC counters can be performed until this bit is not cleared. Note: it is recommended for software to make sure, this bit is cleared after writing '0', before performing any other operation at MAC controller.
CSR21.(8..9)	SCOP	Operation code: 00 – idle state; 01 – read counter; 10 – clear single counter; 11 – clear all counters. After setting command 'clear all counters' none operation on SC registers is possible until end of SC clearing. After clearing all SC counters, SCP bit contains '0' value and it is recommended for software to check if it is cleared after SC clearing, before performing any other operation at MAC controller.
CSR21.(7..0)	SCAD	It contains memory address of counter register. Addresses range are: 0x00 ÷ 0x0F for receive counters; 0x20 ÷ 0x3C for transmit counters.

2.4.21 The SC read value register (CSR22)

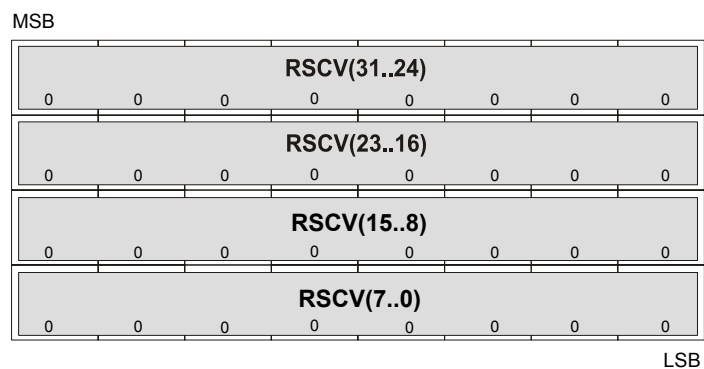


Figure 23. SC read value register

Table 26. SC read value bit functions

Bit	Symbol	Function
CSR22.(31..0)	RSCV	Contains the value of the read SC register.

2.5 AHB interface

The AMBA™ AHB interface is used for the data transfers between the MAC-1G_AMBA and an external shared system memory. It is a master interface; it means that the MAC-1G_AMBA operates as an initiator of the transfers on that interface. The interface operates synchronously with the *hclk* clock supplied by the AHB interface. The data width of the interface can be changed using the AHBDATAWIDTH generic parameter. The values permissible for the AHBDATAWIDTH are 8, 16, or 32.

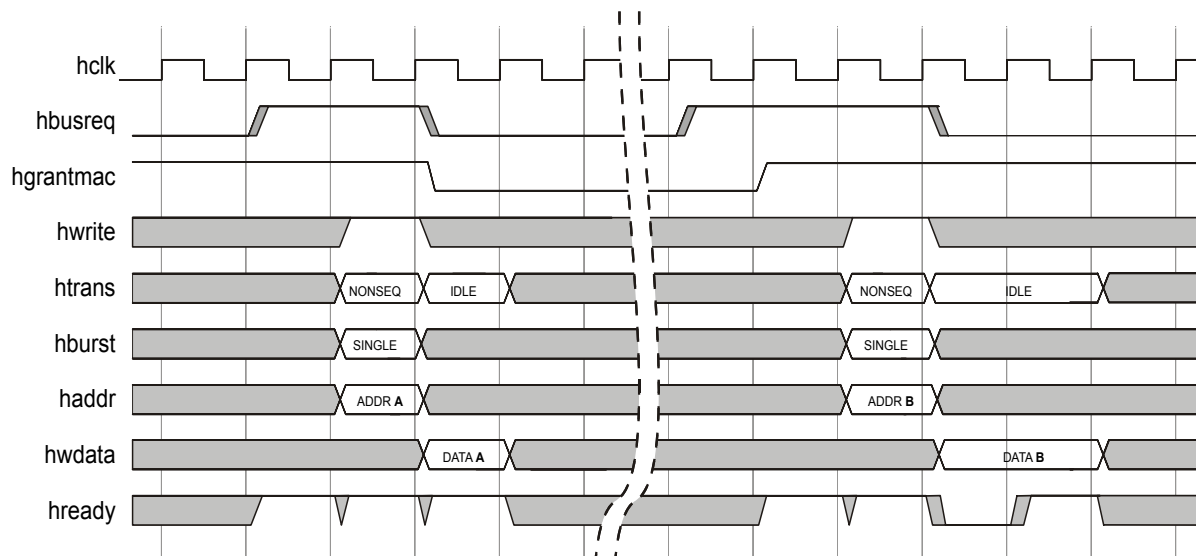


Figure 24. AHB single write operation

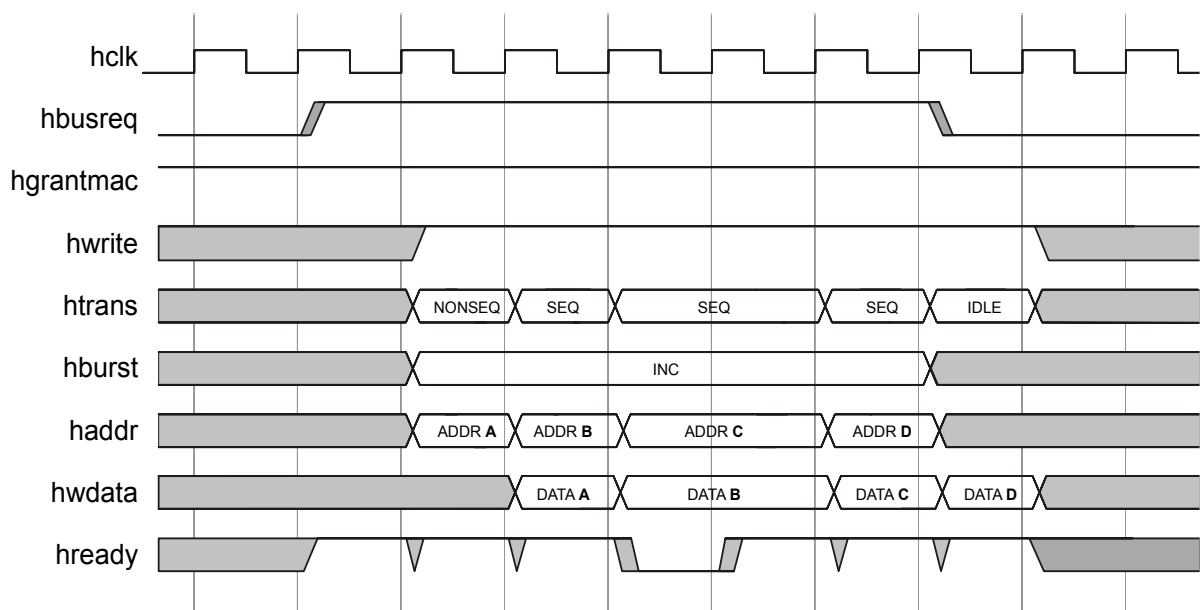


Figure 16. AHB burst write operation

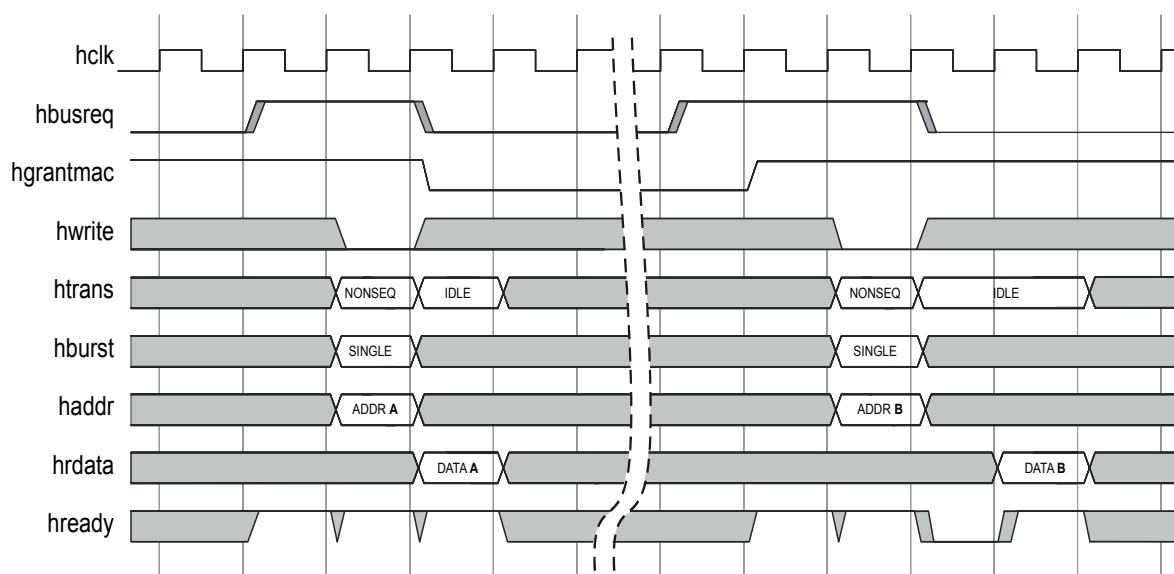
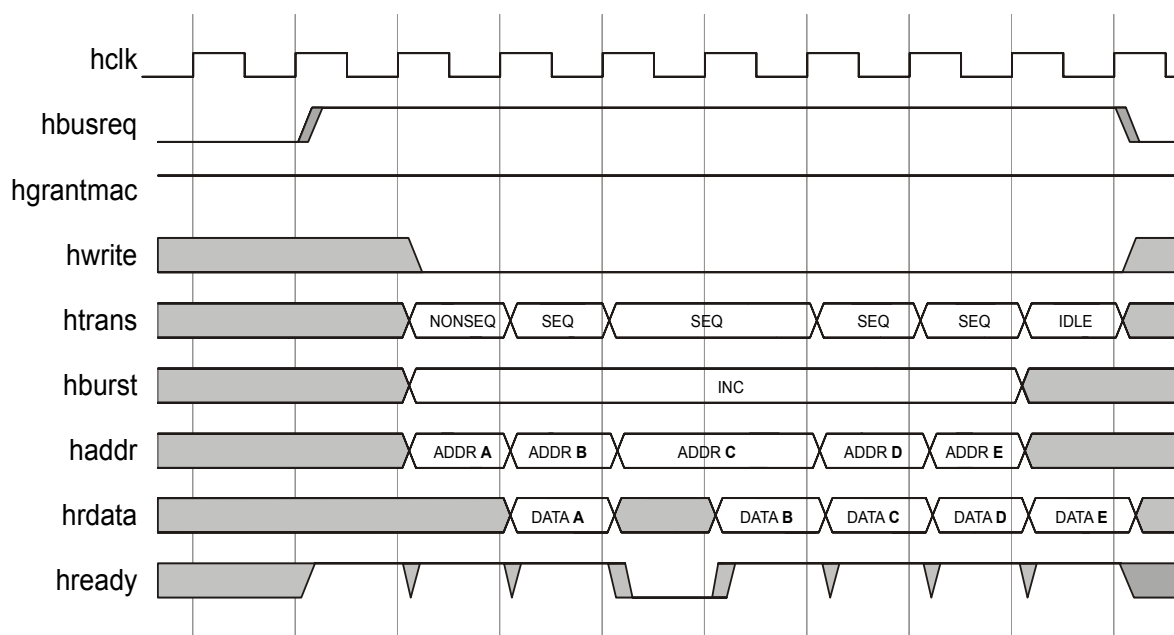
**Figure 17. AHB single read operation****Figure 18. AHB burst read operation**

Table 27. Data interface signals

Name	Type	Polarity/ Bus size	Description
AHB master interface			
hclk	in	rise	AHB bus clock
hresetn	in	low	AHB bus reset
hrdata	in	AHBDATAWIDTH	AHB read data bus
hready	in	high	AHB transfer done
hresp	in	2	AHB slave transfer response
haddr	out	AHBADDRESSWIDTH	AHB bus address
htrans	out	2	AHB transfer type
hwrite	out	1	AHB transfer direction
hsize	out	3	AHB transfer size
hburst	out	3	AHB burst type
hprot	out	4	AHB protection control
hwdata	out	AHBDATAWIDTH	AHB write data
AHB multiple master support			
hgrantmac	in	high	AHB MAC bus grant
hbusreqmac	out	high	AHB MAB bus request
hlockmac	out	high	AHB MAC locked transfer

For more detailed information please refer to AMBA™ AHB specification.

2.6 Frame descriptors

2.6.1 Descriptors/buffers architecture overview

A data exchange between the host and the MAC-1G_AMBA is performed via the descriptor lists and data buffers, which reside in the system shared RAM. The buffers hold the host data to be transmitted or received by the MAC-1G_AMBA. The descriptors act as pointers to these buffers. Each descriptor list should be constructed by the host in a shared memory area, and can be of an arbitrary size. There is a separate list of descriptors for both the transmit and receive processes. The position of the first descriptor in the descriptor list is described by CSR3 for the receive list, and by CSR4 for the transmit list. The descriptors can be arranged in either a "chained" or "ring" structure. In a chained structure every descriptor contains a pointer to the next descriptor in the list. In the ring structure the address of the next descriptor is determined by the CSR0.6..2 (DSL - Descriptor Skip Length). Every descriptor can point to up to two data buffers. When using descriptor chaining the address of the second buffer is used as a pointer to the next descriptor, and thus only one buffer is available. A frame can occupy one or more data descriptors and buffers, but one descriptor cannot exceed a single frame.

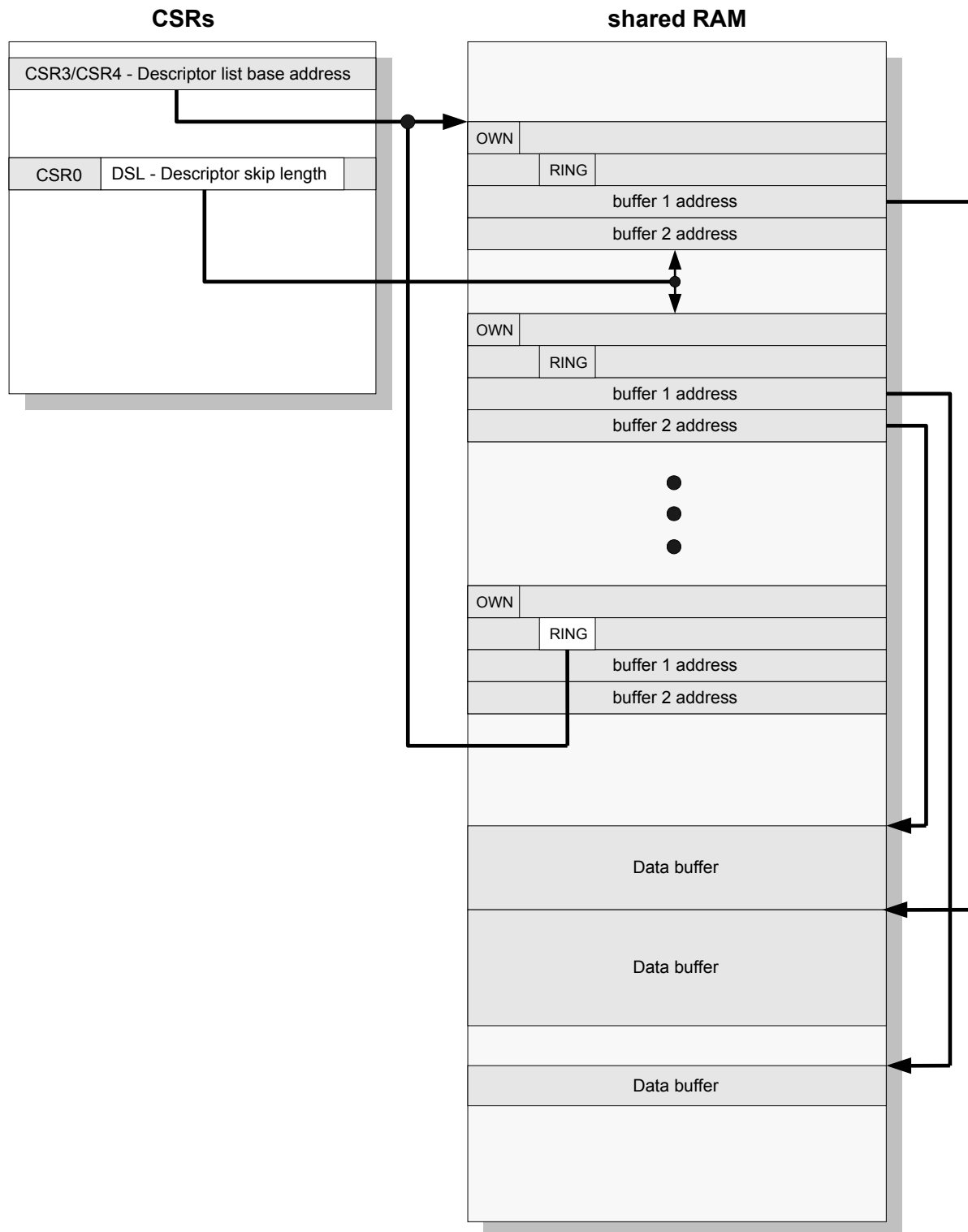


Figure 25. Descriptors in "ring structure"

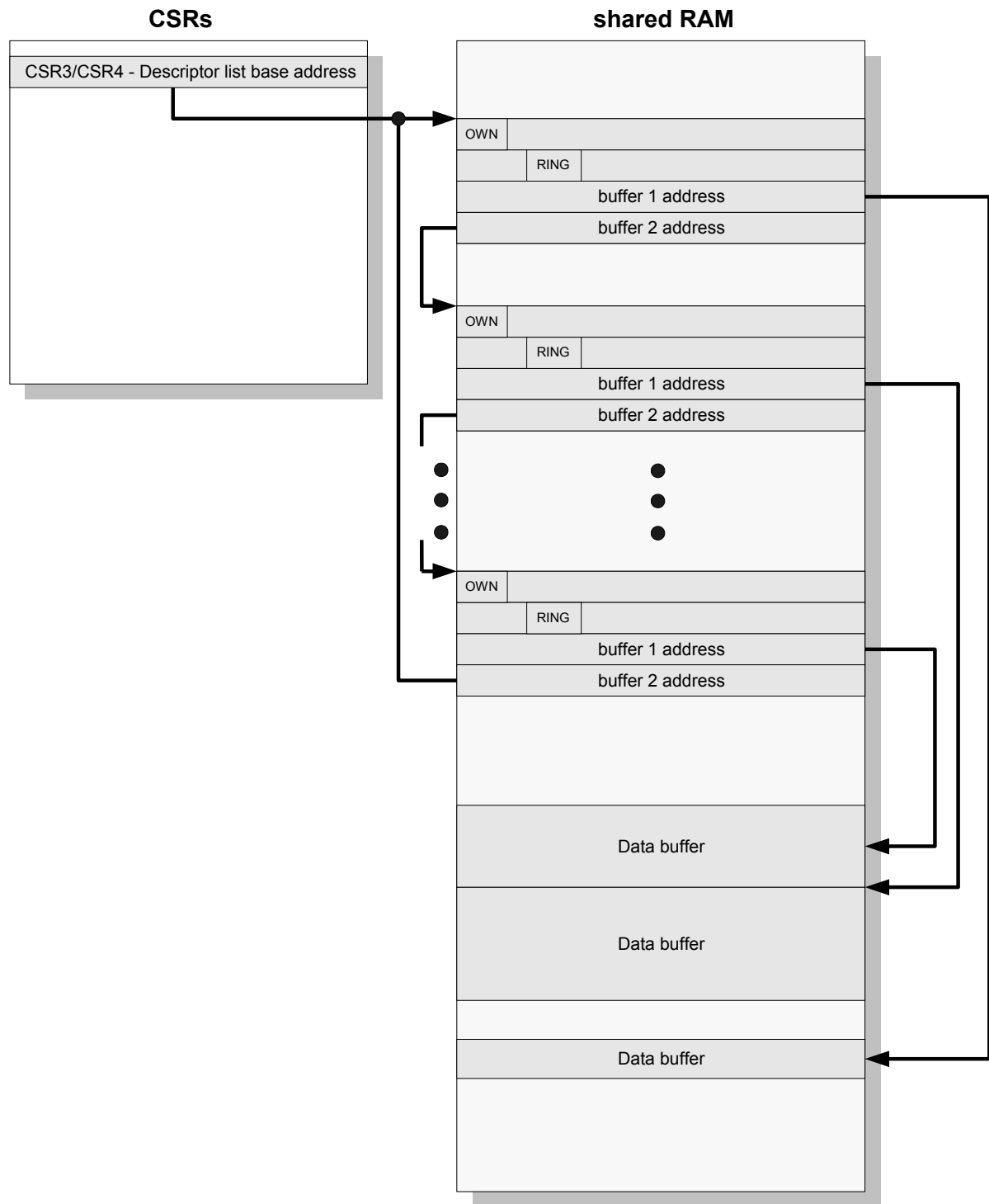


Figure 26. Descriptors in "chained structure"

2.6.2 Receive descriptors

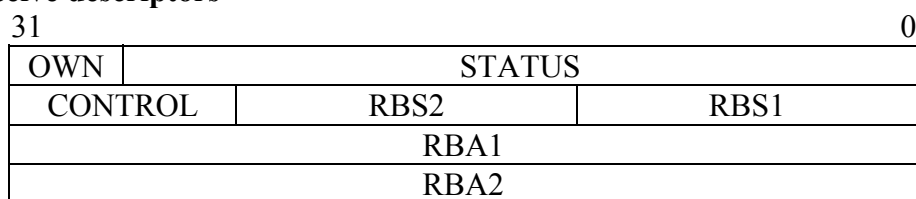


Figure 27. Receive descriptor fields

Table 28. STATUS (RDES0) bit functions

Bit	Symbol	Function
RDES0.31	OWN	Ownership bit. 1 – the MAC-1G_AMBA owns the descriptor. 0 – the host owns the descriptor. The MAC-1G_AMBA will clear this bit when it completes a current frame reception, or when the data buffers associated with a given descriptor are already full.
RDES0.30	FF	Filtering fail. When set, indicates that a received frame did not pass the address recognition process. This bit is valid only for the last descriptor of the frame (RDES0.8 set), when the CSR6.30 (receive all) bit is set, and the frame is at least 64 bytes long.
RDES0.(29..16)	FL	Frame length. Indicates the length, in bytes, of the data transferred into a host memory for a given frame. This bit is valid only when the RDES0.8 (last descriptor) is set and RDES0.14 (descriptor error) is cleared.
RDES0.15	ES	Error summary. This bit is logical or over following bits: RDES0.1 – CRC error. RDES0.6 – Collision seen. RDES0.7 – Frame too long. RDES0.11 – Runt frame. RDES0.14 – Descriptor error. This bit is valid only when the RDES0.8 (last descriptor) is set.
RDES0.14	DE	Descriptor error. Set by the MAC-1G_AMBA when no receive buffer was available when trying to store the received data. This bit is valid only when the RDES0.8 (last descriptor) is set.
RDES0.11	RF	Runt frame. When set, indicates that the frame is damaged by a collision or by a premature termination before the end of a collision window. This bit is valid only when the RDES0.8 (last descriptor) is set.

Bit	Symbol	Function
RDES0.10	MF	Multicast frame. When set, indicates that the frame has a multicast address. This bit is valid only when the RDES0.8 (last descriptor) is set.
RDES0.9	FS	First descriptor. When set, indicates that this is the first descriptor of a frame.
RDES0.8	LS	Last descriptor. When set, indicates that this is the last descriptor of a frame.
RDES0.7	TL	Frame too long. When set, indicates that a current frame is longer than the maximum size of 1518 bytes, as specified by 802.3. This bit is valid only when the RDES0.8 (last descriptor) is set.
RDES0.6	CS	Collision seen. When set, indicates that a late collision was seen (collision after 64 bytes following SFD). This bit is valid only when the RDES0.8 (last descriptor) is set.
RDES0.5	FT	Frame type. When set, indicates that the frame has the length field greater than 1500 (Ethernet type frame). When cleared, indicates the 802.3 type frame. This bit is valid only when the RDES0.8 (last descriptor) is set. Additionally, the FT is invalid for the runt frames of a length shorter than 14 bytes.
RDES0.3	RE	Report on GMII/MII error. When set, indicates that an error has been detected by a physical layer chip connected through the GMII/MII interface. This bit is valid only when the RDES0.8 (last descriptor) is set.
RDES0.2	DB	Dribbling bit. When set, indicates that the frame was not byte aligned. This bit is valid only when the RDES0.8 (last descriptor) is set.
RDES0.1	CE	CRC error. When set, indicates that the CRC error has occurred in the received frame. This bit is valid only when the RDES0.8 (last descriptor) is set. Additionally, the CE is not valid when the received frame is a runt frame.
RDES0.0	ZERO	This bit is reset for the frames with the legal length.

Table 29. CONTROL and COUNT (RDES1) bit functions

Bit	Symbol	Function
RDES1.25	RER	Receive end of ring. When set, indicates that this is the last descriptor in the receive descriptor ring. The MAC-1G_AMBA returns to the first descriptor in the ring, as specified by the CSR3 (Start of receive list address).
RDES1.24	RCH	Second address chained. When set, indicates that the second buffer's address points to the next descriptor and not to the data buffer. Note that the RER takes precedence over the RCH.
RDES1.(21..11)	RBS2	Buffer 2 size. Indicates size, in bytes, of memory space used by the second data buffer. This number must be a multiple of 4. If it is 0, then the MAC-1G_AMBA ignores the second data buffer and fetches the next data descriptor. This number is valid only when the RDES1.24 (Second address chained) is cleared.
RDES1.(10..0)	RBS1	Buffer 1 size. Indicates the size, in bytes, of memory space used by the first data buffer. This number must be a multiple of 4. If it is 0, then the MAC-1G_AMBA ignores the first data buffer and uses the second data buffer.

Table 30. RBA1 (RDES2) bit functions

Bit	Symbol	Function
RDES2.(31..0)	RBA1	Receive buffer 1 address. Indicates the length, in bytes, of memory allocated for the first receive buffer. This number must be longword aligned (RDES2.1..0 = 00).

Table 31. RBA2 (RDES3) bit functions

Bit	Symbol	Function
RDES3.(31..0)	RBA2	Receive buffer 2 address. Indicates the length, in bytes, of memory allocated for the second receive buffer. This number must be longword aligned (RDES3.1..0 = 00).

2.6.3 Transmit descriptors

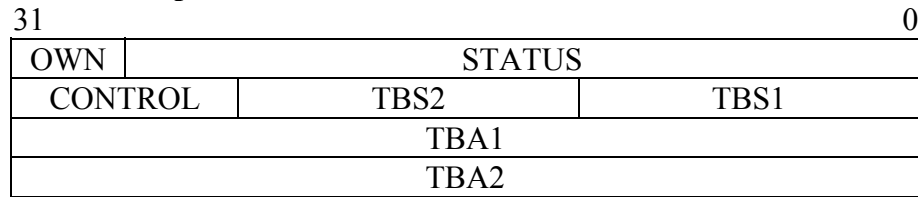


Figure 28. Transmit descriptor fields

Table 32. STATUS (TDES0) bit functions

Bit	Symbol	Function
TDES0.31	OWN	Ownership bit. 1 – the MAC-1G_AMBA owns the descriptor. 0 – the host owns the descriptor. The MAC-1G_AMBA will clear this bit when it completes a current frame transmission or when the data buffers associated with a given descriptor are empty.
TDES0.15	ES	Error summary. This bit is a logical or of the following bits: TDES0.1 – Underflow error. TDES0.8 – Excessive collision error. TDES0.9 – Late collision. TDES0.10 – No carrier. TDES0.11 – Loss of carrier. This bit is valid only when TDES1.30 (last descriptor) is set.
TDES0.11	LO	Loss of carrier. When set, indicates a loss of the carrier during a transmission. This bit is valid only when TDES1.30 (last descriptor) is set.
TDES0.10	NC	No carrier. When set, indicates that the carrier was not asserted by an external transceiver during the transmission. This bit is valid only when TDES1.30 (last descriptor) is set.
TDES0.9	LC	Late collision. When set, indicates that a collision was detected after transmitting 64 bytes. This bit is not valid when TDES0.1 (underflow error) is set. This bit is valid only when TDES1.30 (last descriptor) is set.
TDES0.8	EC	Excessive collisions. When set, indicates that the transmission was aborted after 16 retries. This bit is valid only when TDES1.30 (last descriptor) is set.

Bit	Symbol	Function
TDES0.(6..3)	CC	Collision count. This field indicates the number of collisions that occurred before the end of a frame transmission. This value is not valid when TDES0.8 (Excessive collisions bit) is set. This bit is valid only when TDES1.30 (last descriptor) is set.
TDES0.1	UF	Underflow error. When set, indicates that the FIFO was empty during the frame transmission. This bit is valid only when TDES1.30 (last descriptor) is set.
TDES0.0	DE	Deferred. When set, indicates that the frame was deferred before transmission. Deferring occurs if the carrier is detected when the transmission is ready to start. This bit is valid only when TDES1.30 (last descriptor) is set.

Table 33. CONTROL (TDES1) bit functions

Bit	Symbol	Function
TDES1.31	IC	Interrupt on completion. Setting this flag instructs the MAC-1G_AMBA to set CSR5.0 (transmit interrupt) immediately after processing a current frame. This bit is valid when TDES1.30 (last descriptor) is set, or for a setup packet.
TDES1.30	LS	Last descriptor. When set, indicates the last descriptor of the frame.
TDES1.29	FS	First descriptor. When set, indicates the first descriptor of the frame.
TDES1.28	FT1	Filtering type. This bit together with TDES0.22 (FT0) controls a current filtering mode. This bit is valid only for the setup frames.
TDES1.27	SET	Setup packet. When set, indicates that this is a setup frame descriptor.
TDES1.26	AC	Add CRC disable. When set, the MAC-1G_AMBA does not append the CRC value at the end of the frame. The exception is when the frame is shorter than 64 bytes and automatic byte padding is enabled. In that case the CRC field is added despite the state of the AC flag.
TDES1.25	TER	Transmit end of ring. When set, indicates the last descriptor in the descriptors ring.

Bit	Symbol	Function
TDES1.24	TCH	Second address chained. When set, indicates that the second descriptor's address points to the next descriptor and not to the data buffer. This bit is valid only when TDES1.25 (transmit end of ring) is reset.
TDES1.23	DPD	Disabled padding. When set, automatic byte padding is disabled. The MAC-1G_AMBA normally appends the PAD field after the INFO field when the size of a frame is less than 64 bytes. After padding bytes, the CRC field is also inserted regardless of the state of the AC flag. When the DPD is set, no padding bytes are appended.
TDES1.22	FT0	Filtering type. This bit together with TDES0.28 (FT1), controls the current filtering mode. This bit is valid only when TDES1.27 (SET) bit is set.
TDES1.(21..11)	TBS2	Buffer 2 size. Indicates the size, in bytes, of memory space used by the second data buffer. If it is zero, the MAC-1G_AMBA ignores the second data buffer and fetches the next data descriptor. This bit is valid only when TDES1.24 (second address chained) is cleared.
TDES1.(10..0)	TBS1	Buffer 1 size. Indicates the size, in bytes, of memory space used by the first data buffer. If it is 0, the MAC-1G_AMBA ignores the first data buffer and uses the second data buffer.

Table 34. TBA1 (TDES2) bit functions

Bit	Symbol	Function
TDES2.(31..0)	TBA1	Transmit buffer 1 address. Contains the address of the first data buffer. For the setup frame this address must be longword aligned (TDES3.1..0 = 00). In all other cases there are no restrictions on buffer alignment.

Table 35. TBA2 (TDES3) bit functions

Bit	Symbol	Function
TDES3(31..0)	TBA2	Transmit buffer 2 address. Contains the address of the second data buffer. There are no restrictions on buffer alignment.

2.6.4 Setup frames

The setup frames define the addresses which are used for the receive address filtering process. These frames are never transmitted on the ethernet. They are used to fill the address-filtering RAM. A valid setup frame must be exactly 192 bytes long, and must be allocated in a single buffer that is longword aligned. Also, both TDES1.29 (First descriptor) and TDES1.30 (Last descriptor) must be cleared for the setup frame. The FT1 and FT0 bits of the setup frame define the current filtering mode. Table 28 lists all possible combinations. Table 29 shows the setup frame buffer format for perfect filtering modes, and Table 31 for imperfect filtering modes.

Table 36. Filtering type selection

FT1	FT0	Description
0	0	Perfect filtering mode. Setup frame buffer is interpreted as a set of 16 48-bit physical addresses.
0	1	Hash filtering mode. Setup frame buffer contains 512-bit hash table plus a single 48-bit physical address.
1	0	Inverse filtering mode. Setup frame buffer is interpreted as a set of 16 48-bit physical addresses.
1	1	Hash only filtering mode. Setup frame buffer is interpreted as a 512-bit hash table.

Table 37. Perfect filtering setup frame buffer format

byte #	data bits 31:16	data bits 15:0
3:0	xxxxxxxxxxxxxxxx	Physical Address 0 (15:00)
7:4	xxxxxxxxxxxxxxxx	Physical Address 0 (31:16)
11:8	xxxxxxxxxxxxxxxx	Physical Address 0 (47:32)
12:9	xxxxxxxxxxxxxxxx	Physical Address 1 (15:00)
16:13	xxxxxxxxxxxxxxxx	Physical Address 1 (31:16)
20:17	xxxxxxxxxxxxxxxx	Physical Address 1 (47:32)
*	*	*
*	*	*
*	*	*
	xxxxxxxxxxxxxxxx	Physical Address 14 (15:00)
	xxxxxxxxxxxxxxxx	Physical Address 14 (31:16)
	xxxxxxxxxxxxxxxx	Physical Address 14 (47:32)
183:180	xxxxxxxxxxxxxxxx	Physical Address 15 (15:00)
187:184	xxxxxxxxxxxxxxxx	Physical Address 15 (31:16)
191:188	xxxxxxxxxxxxxxxx	Physical Address 15 (47:32)

Table 38. Hash table setup frame buffer format

byte #	data bits 31:16	data bits 15:0
3:0	xxxxxxxxxxxxxxxx	Hash filter (015:000)
7:4	xxxxxxxxxxxxxxxx	Hash filter (031:016)
11:8	xxxxxxxxxxxxxxxx	Hash filter (047:032)
* * *		
123:121	xxxxxxxxxxxxxxxx	Hash filter (495:480)
127:124	xxxxxxxxxxxxxxxx	Hash filter (511:496)
131:128	xx	
135:132	xx	
* * *		
159:156	xxxxxxxxxxxxxxxx	Physical Address (15:00)
163:160	xxxxxxxxxxxxxxxx	Physical Address (31:16)
167:164	xxxxxxxxxxxxxxxx	Physical Address (47:32)
171:168	xx	
175:172	xx	
* * *		
183:180	xxxxxxxxxxxxxxxx	xxxxxxxxxxxxxxxx
187:184	xxxxxxxxxxxxxxxx	xxxxxxxxxxxxxxxx
191:188	xxxxxxxxxxxxxxxx	xxxxxxxxxxxxxxxx

2.7 External dual-port RAM interface

2.7.1 Obligatory RAM interfaces

The MAC-1G_AMBA provides three external dual port RAM interfaces: the first for the transmit data, the second for the receive data, and the third for the receive address-filtering block. For proper operation of the core, the system designer should supply synchronous dual port RAM memories.

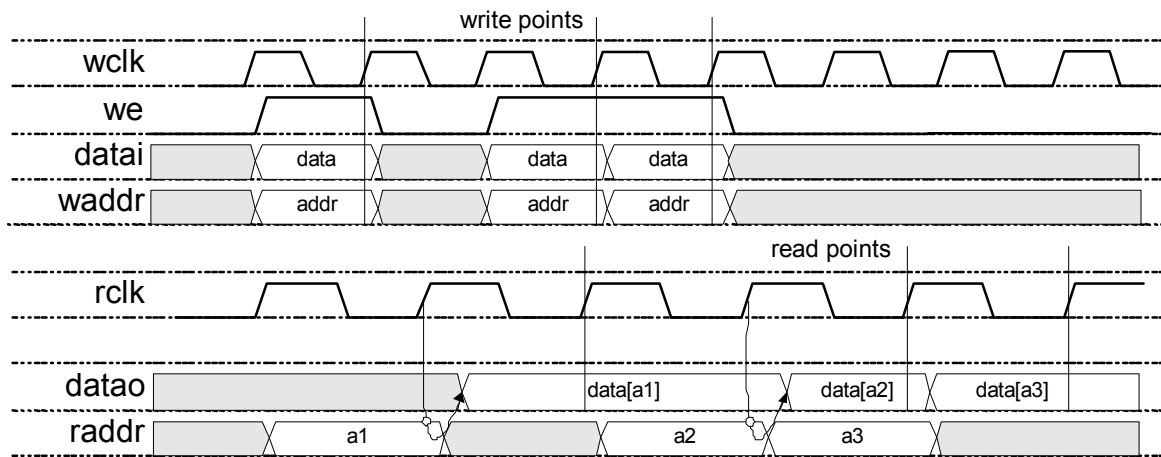


Figure 29. Dual port RAM write and read operations

Table 39. Dual port RAM pins mapping

Dual port RAM pin name	Pin function	Transmit dual port RAM	Receive dual port RAM	Address dual port RAM
wclk	write clock	clkdata	clkr	clkdata
we	write enable	twe	rwe	fwe
rdata	read data	trdata	rrdata	frdata
waddr	write address	twaddr	rwaddr	fwaddr
rclk	read clock	clkt	clkdata	clkr
raddr	read address	traddr	rraddr	fraddr
wdata	write data	twdata	rwdata	fwdata

Table 40. Dual-port RAM parameters

Parameter		Receive dual port RAM	Transmit dual port RAM	Address dual port RAM
Data bus width		AHBDATAWIDTH	AHBDATAWIDTH	16
Address bus width	min	6	6	6
	typ	9 (2kB/32bit)	9 (2kB/32bit)	6
	max	16	16	6

The address bus width of the Receive/Transmit dual port RAMs can be customized via the RFIFODEPTH/TFIFODEPTH generic parameters. Those memories should be at least as big as the longest frame used on a given network.

2.7.2 RAM interfaces for optional statistical counters module

The MAC-1G_AMBA may provide two additional external dual port RAM interfaces for supporting SC module. For proper operation of the core, the system designer should supply synchronous dual port RAM memories that have possibility of reading and writing at both ports.

All data buses of SC dual RAM are 32-bit width. The ar bus signal is 4-bit wide and the at bus signal is 5-bit wide. These buses width can not be customized and they are fixed.

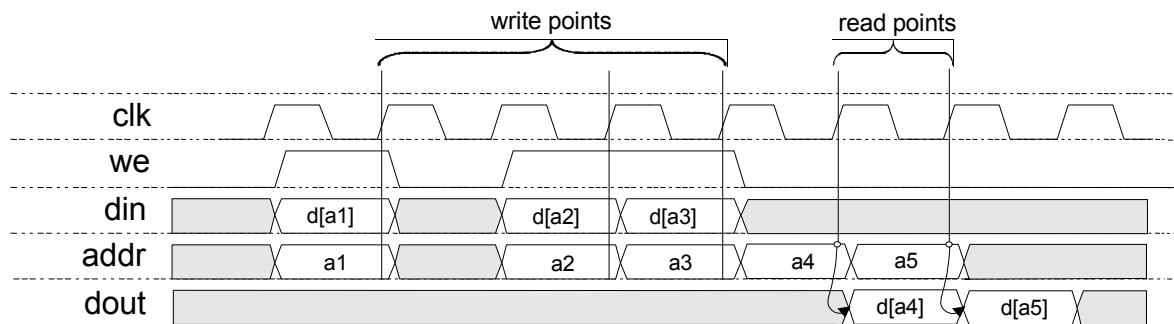


Figure 30. SC dual port RAM write and read operations

Table 41. SC dual port RAM pins mapping

Dual port RAM pin name	Pin function	Receive SC dual port RAM		Transmit SC dual port RAM	
		host side	network side	host side	network side
clk	clock	pclk	hclk	pclk	hclk
we	write enable	sscclrr	ramwer	sscclrt	ramwet
din	read data	zero	dir	zero	dit
addr	write address	scadr	ar	scadt	at
dout	write data	schr	dor	schr	dot

2.8 Statistical counters controller

Statistical counters module relieves the host and software drivers of the responsibility of maintaining statistics for managed Ethernet port. SC module is responsible for:

- handling receive and transmit events, i.e. incrementing appropriate counters;
- blocking the possibility of writing to SC register, when read operation is performed by host via CSR registers;
- clearing all SC registers after setting 'clear all SC' command.

There is implemented a simply form of handshaking, between SC and CSR module. It prevents the situation when the one counter is read by CSR module in the moment when it is incremented by SC module. During any kind of operation on SC registers, they values of counters are blocked and none of them is incremented, even when there is a frame to handle.

The statistical counters are located physically in the MAC-1G_AMBA core and can be accessed directly by a host via CSR21 and CSR22 registers, described in 2.4.20 and 2.4.21 chapters.

All the SC are 32 bits long and quad word-aligned. If counter reaches 0xFFFFFFFF, it wraps around to zero.

For each successfully transmitted packet, the Statistical Counters block updates four or five counters:

- TxFrmOK;
- TxOctOK;
- One of TxUniOK, TxMultiOK or TxBroadOK;
- One of collision frames counters (Coll0, Coll1...Coll15) or TxDefer;
- Zero or TxPause.

For each attempted transmission that ends in error, the Statistical Counters block updates one of Coll16 counters (excessive collision): TxLCErr, TxMACErr or TxCSErr.

For each successfully received packet, the Statistical Counters block updates four to five counters:

- RxFrmOK;
- RxOctOK;
- One of RxUniOK, RxMultiOK or RxBroadOK;
- One of ReceivePacketSizes (1to6): Rx1024to1518, Rx512to1023, Rx256to51, Rx128to255, Rx65to127 or Rx64;
- Zero or RxPause.

For each packet received that ends in error or rejection, the Statistical Counters block updates only one of RxTooLong, Rx63orLess, RxAlignErr or RxFCSErr counters

There are two possibilities of zeroing all statistical counters.

- writing 'clear all SC' command to the CSR21 register.
- Software (CSR0.SWR) or hardware reset of the whole MAC component

During zeroing SC, none operation on SC registers is possible until end of SC clearing. After clearing all SC counters, both SCOP and SCP field of CSR21 register contains respectively '00' and '0' values.

Table 42. Transmit statistical counters description

Transmit counter name	Address (H)	Description
TxFrmOK	0x20	This counter is incremented for each frame successfully transmitted. This count includes broadcast, multicast and unicast address frames.
TxOctOK	0x21	This counter is incremented by the number of bytes transmitted after each successfully transmitted frame. The frame byte count includes addresses, type, data and FCS field.
TxUniOK	0x22	This counter is incremented for each unicast frame successfully transmitted.
TxMultiOK	0x23	This counter is incremented for each multicast frame successfully transmitted that is not broadcast address.
TxBroadOK	0x24	This counter is incremented for each broadcast frame successfully transmitted.
Coll [0÷15]	0x25÷ 0x34	This is vector of 16 counters, each recording the number of frames successfully transmitted which experienced 0, 1, 2...15 collisions. For 0 collisions, either FramesWithDeferredTransmission or CollisionFrame_0 is incremented, depending on whether deferral occurred. The address of one collision counter is: CollisionFrame_N address = 0x94 + N*4. Some related counters are defined as follows: NoCollisionFrames= CollisonFrames_0+ FramesWithDeferredTransmission SingleCollision=CollisionFrame_1 MultipleCollison=sum of CollisionFrame (2 to15)
Coll16	0x35	This counter is incremented for each transmission aborted after experiencing 16 in-window collisions.
TxDefer	0x36	This counter is incremented for each successfully transmitted frame with no collision, which is experienced delay because medium was busy.
TxPause	0x37	This counter is incremented for each successfully transmitted MAC control frames with pause operand.
TxLCErr	0x3A	This counter is incremented for each frame which experienced a late collision.
TxMACErr	0x3B	This counter is incremented for each frame aborted due to underrun frame. If this counter is incremented, none of error counter is incremented.
TxCSErr	0x3C	This counter is incremented for each frame, which experienced no carrier sense or dropped carrier sense during transmission.

Table 43. Receive statistical counters description

Transmit counter name	Address (H)	Description
RxFrmOK	0x00	Counter incremented for each successfully transmitted packet.
RxOctOK	0x01	Counter incremented by the number of bytes transmitted after each successfully transmitted packet.
RxUniOK	0x02	Counter incremented for each frame successfully transmitted to unicast address.
RxMultiOK	0x03	Counter incremented for each frame successfully transmitted to multicast address that is not broadcast address.
RxBroadOK	0x04	Counter incremented for each frame successfully transmitted to broadcast address.
RxFrmSizes(0:7)	0x05÷ 0x0C	An array of eight 32-bit counters. Each counter counts good packets received, grouped by packet size. The index values are calculated as follows: FrTooLongErrCnt – counter incremented for each frames that more than 1518 octets; Fr1024to1518OctCnt - counter incremented for each frames that contains 1024 to 1517 octets; Fr512to1023OctCnt - counter incremented for each frames that contains 512 to 1023 octets; Fr256to511OctCnt - counter incremented for each frames that contains 256 to 511 octets; Fr128to255OctCnt - counter incremented for each frames that contains 128 to 255 octets; Fr65to127OctCnt - counter incremented for each frames that contains 65 to 127 octets; Fr64OctetsCnt - counter incremented for each frames that contains exactly 64 octets; Fr63orLessCnt - counter incremented for each frame that contains 63 or less octets.
RxPause	0x0D	Counter incremented for each MAC Pause control frames successfully received.
RxAlignErr	0x0E	Counter incremented for each frame that are not an integral number of octets in length and do not pass the FCS check.
RxFCSErr	0x0F	Counter incremented for each frame that are an integral number of octets in length and do not pass the FCS check.

2.9 DMA controller

The DMA (Direct Memory Access) is used to control a data flow between the host and the MAC-1G_AMBA.

The DMA services the following types of requests from the MAC-1G_AMBA's transmit and receive processes:

- Transmit request:
 - Descriptor fetch
 - Descriptor closing
 - Setup packet processing
 - Data transfer from host buffer to transmit the FIFO
- Receive request
 - Descriptor fetch
 - Descriptor closing
 - Data transfer from receive the FIFO to the host buffer

The key task for the DMA is to perform an arbitration between the receive and transmit processes. Two arbitration schemes are possible, selected by the CSR0.1 bit:

- 1 – Round-robin arbitration scheme in which two processes have equal priorities
- 0 – The receive process has priority over the transmit process unless transmission is in progress.

In this case the following rules apply:

- The transmit process request should be serviced by the DMA between two consecutive receive transfers
- The receive process request should be serviced by the DMA between two consecutive transmit transfers

Transfers between the host and the MAC-1G_AMBA performed by the DMA component are either single data transfers or "burst" transfers. For the data descriptors the data transfer size depends on the DATAWIDTH generic parameter. The rule is that every descriptor field (32-bit) is accessed with a single burst. For the DATAWIDTH=32, the descriptors are accessed with a single transaction, for the DATAWIDTH=16, the descriptors are accessed with a burst of two 16-bit words, and for the DATAWIDTH=8, the descriptors are accessed with a burst of four 8-bit words.

In the case with data buffers, the burst length is defined by CSR0(13..8) (Programmable burst length), and can be set to 0, 1, 2, 4, 8, 16, or 32. When set to 0, no maximum burst size is defined and the transfer ends when the transmit FIFOs are full or the receive FIFOs are empty.

2.10 Transmit process

The transmit process can operate in one of three modes: running, stopped or suspended. After a software or hardware reset, or after a stop transmit command, the transmit process is in a stopped state. The transmit process can leave a stopped state only after the start transmit command.

When in a running state the transmit process performs descriptor/buffer processing. When operating in a suspended or stopped state the transmit process retains the position of the next descriptor i.e., the address of the descriptor following the last descriptor being closed. After entering a running state, that position is used for the next descriptor fetch. The only exception is when the host writes the transmit descriptor base address register (CSR4). In that case the descriptor address is reset and the fetch is directed to the first position in the list.

The transmit process remains running until one of the following events occur:

- The hardware or software reset is issued. Setting the CSR0.0 (SWR) bit can initiate the software reset. After the reset, all internal registers return to their default states. The current descriptor's position in the transmit descriptors list is lost.
- A stop transmit command is issued by the host. This can be performed by writing 0 to the CSR6.13 (ST) bit. The current descriptor's position is retained.
- The descriptor owned by the host is found. A current descriptor's position is retained.
- The transmit FIFO underflow error is detected. An underflow error is generated when the transmit FIFO is empty during the transmission of the frame. When it occurs, the transmit process enters a suspended state. Transmit automatic polling is internally disabled, even if it is enabled by the host by writing the TAP bits. The current descriptor's position is retained.

Leaving a suspended state is possible in one of the following situations:

- A transmit poll demand command is issued. This can be performed by writing CSR1 with nonzero value. The transmit poll demand command can be also generated automatically when Transmit automatic polling is enabled. Transmit automatic polling is enabled only if the CSR0(19..17) (TAP) bits are written with a nonzero value, and when there was no underflow error prior to entering the suspended state.
- A stop transmit command is issued by the host. This can be performed by writing a 0 to the CSR6.13 (ST) bit. The current descriptor's position is retained.

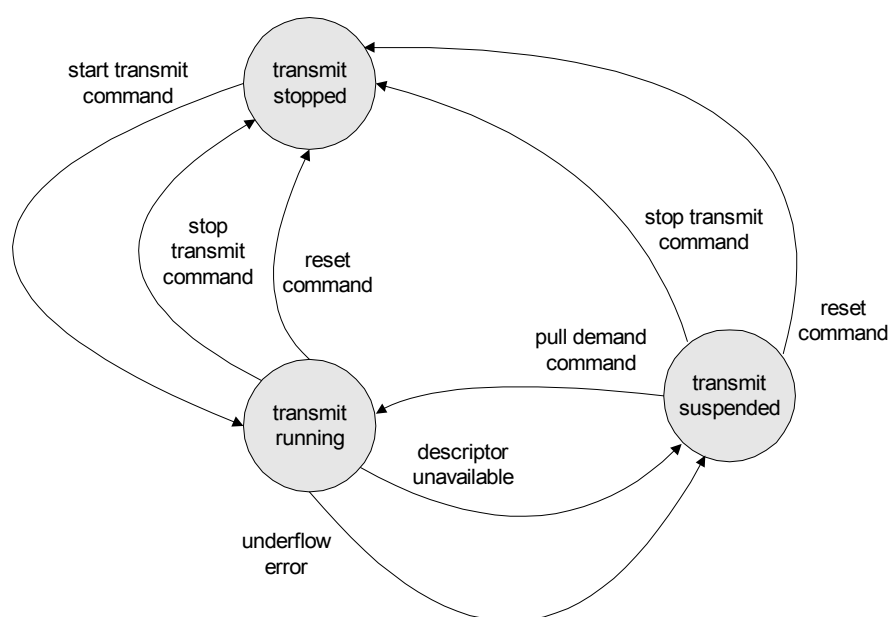


Figure 31. Transmit Process transitions

2.11 Receive process

The receive process can operate in one of three modes: running, stopped or suspended. After a software or hardware reset, or after a stop receive command, the receive process is in the stopped state. The receive process can leave a stopped state only after a start receive command.

In the running state the receiver performs descriptor/buffer processing. In the running state the receiver fetches from the receive descriptor list. It performs this fetch regardless of whether there is any frame on the link. When there is no frame pending, the receive process reads the descriptor and simply waits for the frames. When a valid frame is recognized, the receive process starts to fill the memory buffers pointed to by the current descriptor. When the frame ends, or when the memory buffers are completely filled, the current frame descriptor is closed (ownership bit cleared). Immediately, the next descriptor on the list is fetched in the same manner, and so on.

When operating in a suspended or stopped state, the receive process retains the position of the next descriptor i.e., the address of the descriptor following the last descriptor being closed. After entering a running state that position is used for the next descriptor fetch. The only exception is when the host writes the receive descriptor base address register (CSR3). In that case the descriptor address is reset and the fetch is pointed to the first position in the list.

The receive process remains running until one of the following events occur:

- A hardware or software reset is issued by the host. A software reset can be performed by setting the CSR0.0 (SWR) bit. After reset, all internal registers return to their default states. The current descriptor's position in the receive descriptors list is lost.
- A stop receive command is issued by the host. This can be performed by writing a 0 to the CSR6.1 (SR) bit. The current descriptor's position is retained.
- The descriptor owned by the host is found by the MAC-1G_AMBA during the descriptor fetch. The current descriptor's position is retained.

Leaving a suspended state is possible in one of the following situations:

- A receive poll command is issued by the host. This can be performed by writing a nonzero value to CSR2.
- A new frame is detected by the MAC-1G_AMBA on a receive link.
- A stop receive command is issued by the host. This can be performed by writing a 0 to the CSR6.1 (SR) bit. The current descriptor's position is retained.

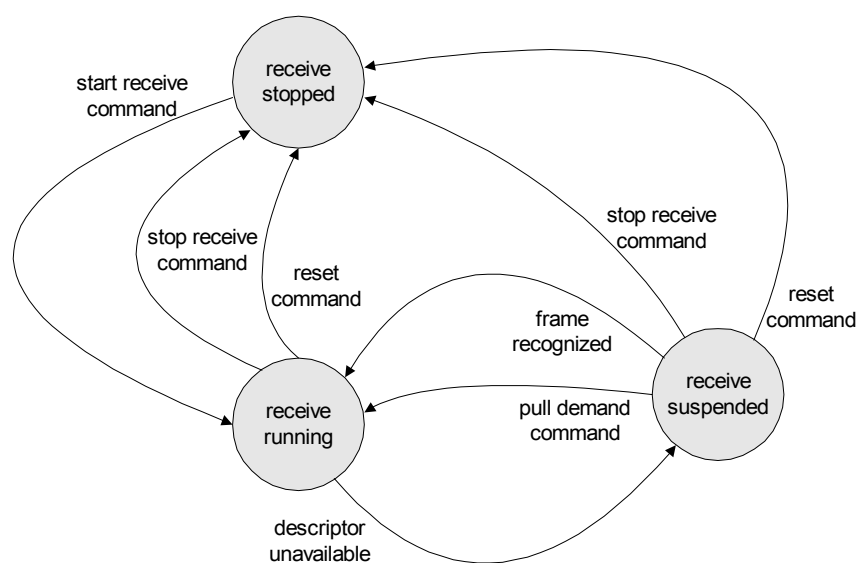


Figure 32. Receive Process transitions

2.12 Interrupt control

The interrupt controller uses three internal Control and Status Registers: CSR5, CSR7 and CSR11. The CSR5 contains the MAC-1G_AMBA's status information. It has 10 bits that can trigger an interrupt. These bits are collected in two groups: normal interrupts, and abnormal interrupts. Each group has its own summary bit, the NIS and AIS, respectively. The NIS and AIS bits directly control the *int* pin of the MAC-1G_AMBA. Every status bit in the CSR5 that can source an interrupt can be individually masked by writing an appropriate value into the CSR7 - Interrupt Enable Register.

An interrupt mitigation mechanism is provided for reducing the CPU usage in servicing interrupts. The interrupt mitigation is controlled via the CSR11. There are separate interrupt mitigation control blocks for the transmit and receive interrupts. Both of these blocks consist of a 4-bit frame counter and a 4-bit timer. The operation of these blocks is similar for the receive and transmit processes. After the end of a successful receive or transmit operation, an appropriate counter is decremented and the timer starts to count down, if it has not already started. An interrupt is triggered when either the counter or timer reaches zero. This provides the possibility to generate a single interrupt for a few received/transmitted frames, or after a specified time from the last successful receive/transmit operation.

It is possible to omit the transmit interrupt mitigation for one particular frame by setting the IC (Interrupt on Completion) bit in the last descriptor of the frame. If the IC bit is set, the MAC-1G_AMBA sets transmit interrupt immediately after the frame has been transmitted.

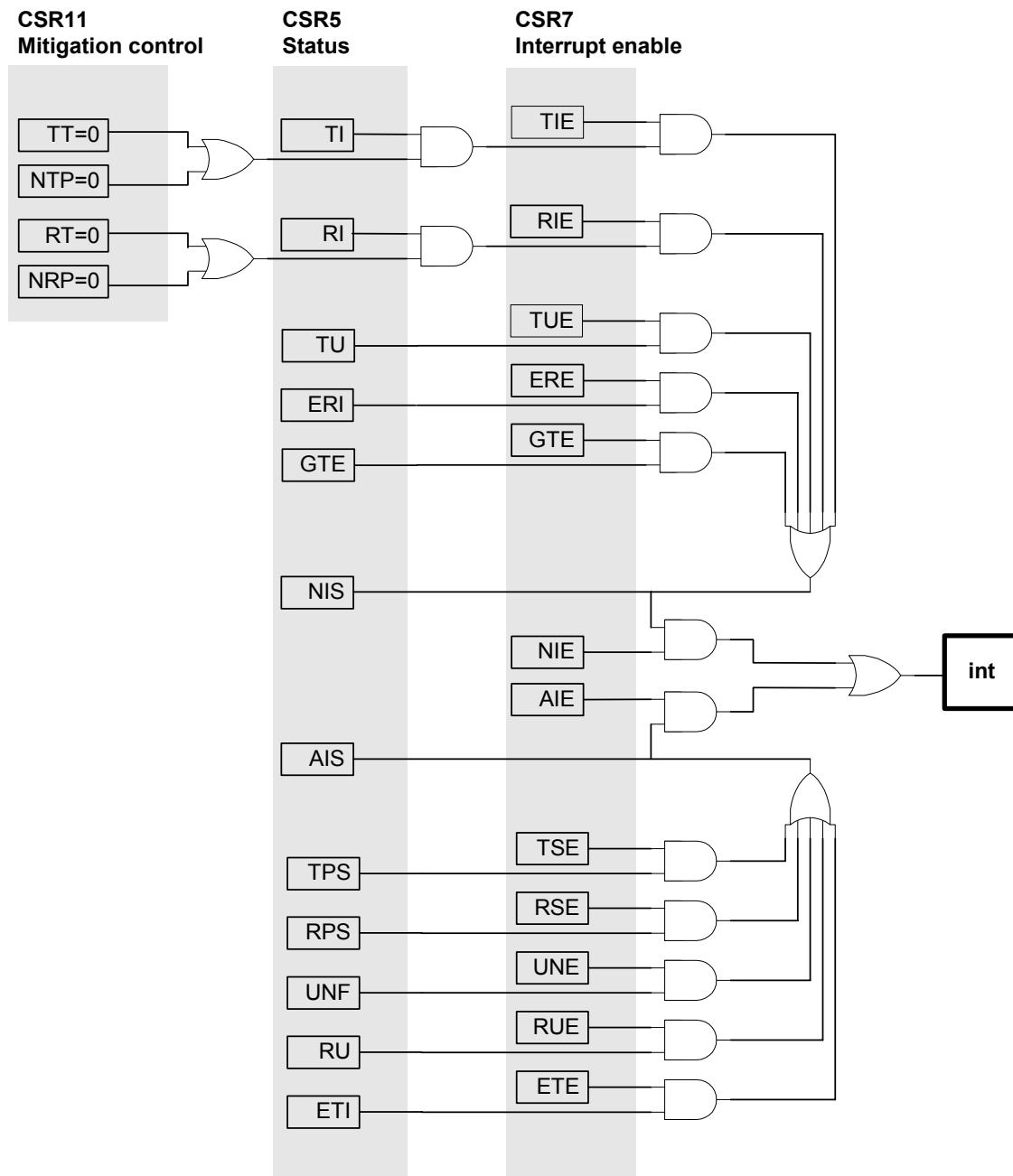


Figure 33. Interrupt scheme

2.13 General purpose timer

A 16-bit general-purpose timer is provided to simplify host calculated time intervals. The timer operates synchronously with the transmit clock *clkt* generated by the PHY device. This gives the host the possibility of measuring the time intervals based on actual ethernet bit time.

The timer can operate in a "one shot" mode or a continuous mode. In a "one shot" mode the timer stops after reaching zero, and in a continuous mode, upon reaching zero, the timer is automatically reloaded and continues counting.

The actual count value can be tested with an accuracy of ± 1 bit by reading CSR11(15..0). When writing to CSR11(15..0), the data are stored in the internal reload register. The timer is immediately reloaded and starts to count down.

2.14 Data link layer operation

2.14.1 GMII/MII interface

The MAC-1G_AMBA uses a standard GMII/MII interface as defined in the 802.3 standard. This interface can be used for connecting the MAC-1G_AMBA to an external Ethernet 10/100/1000 PHY device.

If the 1G speed is used, then the MAC-1G_AMBA will use all data bits of the GMII/MII data interface (*txd(7..0)*, *rx(7..0)*). If the 10 or 100 Mbit speed is used, then the MAC-1G_AMBA will use only four least significant bits of the GMII/MII data interface (*txd(3..0)*, *rx(3..0)*). Additionally, *gbo* (Gigabit selection) output is active when the MAC-1G_AMBA operates with 1G speed.

Table 44. External PHY interface pins

IEEE 802.3 signal name	MAC-1G_AMBA signal name	Description
RX_CLK	clkr	Clock for receive operation. This should be: 125 MHz clock for a 1000Mb/s operation, 25 MHz clock for a 100Mb/ operation, 2.5 MHz clock for a 10Mb/s operation.
RX_DV	rx(7..0)	Receive data valid signal. The PHY device should assert the <i>rx(7..0)</i> when a valid data nibble is provided on <i>rx(7..0)</i> . <i>rx(7..0)</i> must be synchronous with the <i>clkr</i> receive clock.
RX_ER	rxer	Receive error. The MAC-1G_AMBA ends a reception when this bit is asserted during a receive operation. <i>rxer</i> must be synchronous with the <i>clkr</i> receive clock.
RXD	rx(7..0)	Receive data recovered and decoded by PHY. The <i>rx(7..0)</i> is the least significant bit. The <i>rx(7..0)</i> must be synchronous to the <i>clkr</i> receive clock. When operating in 10/100Mb/s MII mode, bits <i>rx(7..4)</i> are ignored by the MAC-1G_AMBA.
TX_CLK	clkt	Clock for transmit operation. This should be: 125 MHz clock for a 1000Mb/s operation, 25 MHz clock for a 100Mb/ operation, 2.5 MHz clock for a 10Mb/s operation.
TX_EN	txen	Transmit enable. When asserted indicates valid data for the PHY on the <i>tx(7..0)</i> . <i>txen</i> is synchronous with the <i>clkt</i> transmit clock.

IEEE 802.3 signal name	MAC-1G_AMBA signal name	Description
TXD	txd	Transmit data. <i>txd(0)</i> is the least significant bit. <i>txd</i> is synchronous with the <i>clkt</i> transmit clock.
COL	col	Collision detected. This signal should be asserted by the PHY when a collision is detected. It is valid only when operating in a half duplex mode. When operating in full duplex mode this signal is ignored by the MAC-1G_AMBA. <i>col</i> is not required to be synchronous to either <i>clkr</i> or <i>clkt</i> . <i>col</i> is sampled internally by the <i>clkt</i> clock.
CRS	crs	Carrier sense. This signal should be asserted by the PHY when either a receive or transmit medium is non-idle. <i>csr</i> is not required to be synchronous to either <i>clkr</i> or <i>clkt</i> .
TX_ER	-	Transmit error. Not used by the MAC-1G_AMBA.
MDC	mdc	GMII/MII management clock. This signal is driven by the CSR9.16 bit.
MDIO	mdi	GMII/MII management data input. The state of this signal can be checked by reading the CSR9.19 bit.
	mdo	GMII/MII management data output. This signal is driven by the CSR9.18 bit.

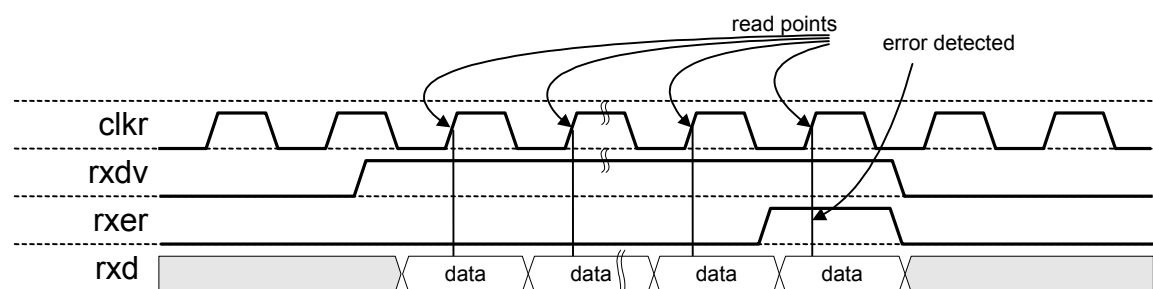


Figure 34. GMII/MII receive operation

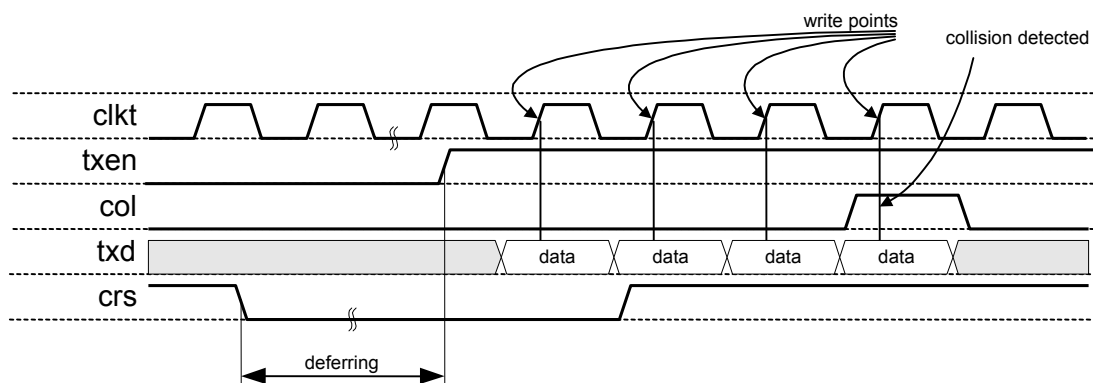


Figure 35. GMII/MII transmit operation

2.14.2 Frame format

The MAC-1G_AMBA supports the Ethernet frame format shown on Figure 27. The standard Ethernet frames (DIX Ethernet), as well as the IEEE 802.3 frames are accepted. The MAC-1G_AMBA processes the individual frame fields according to Table 34.

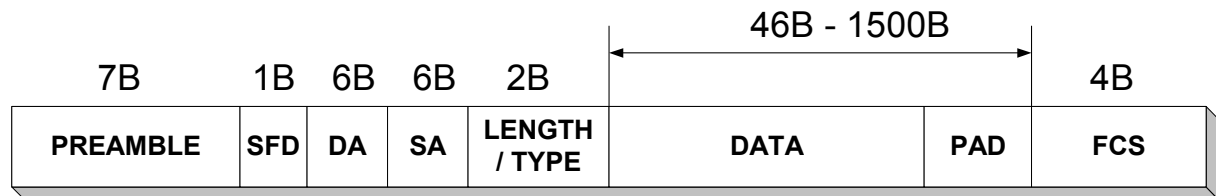


Figure 36. 802.3 Frame format

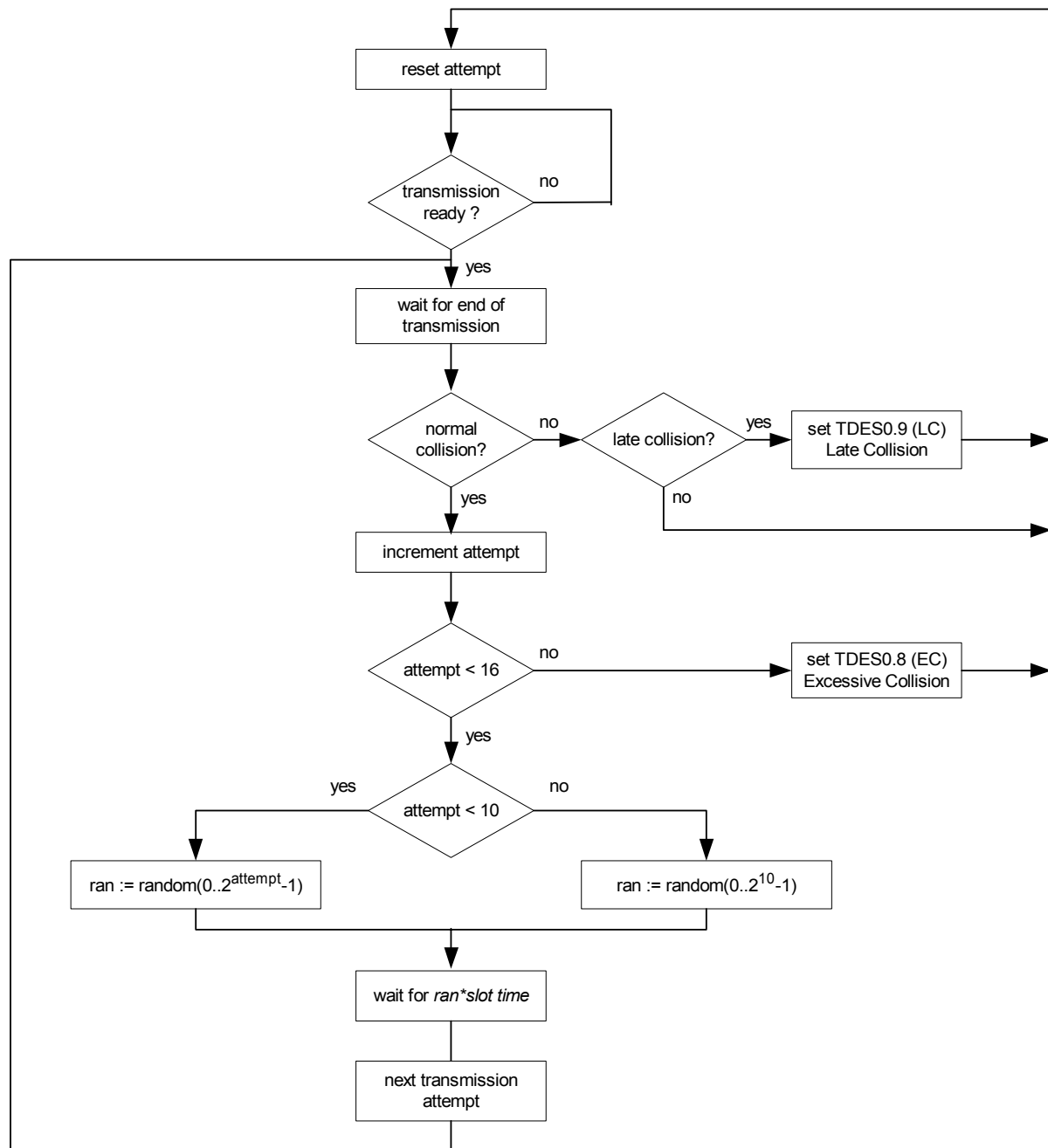
Table 45. Frame fields usage

Field	Width (Bytes)	Transmit operation	Receive operation
PREAMBLE	7	Generated by the MAC-1G_AMBA.	Stripped from received data. Not required for proper operation.
SFD	1	Generated by the MAC-1G_AMBA	Stripped from received data.
DA	6	Supplied by the host.	Checked by the MAC-1G_AMBA according to the current address filtering mode and passed to the host.
SA	6	Supplied by the host.	Passed to the host.
LENGTH / TYPE	6	Supplied by the host.	Passed to the host.
DATA	0-1500	Supplied by the host.	Passed to the host.
PAD	0-46	Generated by the MAC-1G_AMBA when the CSR.23(DPD) bit is cleared and data supplied by the host is less then 64 bytes.	Passed to the host.
FCS	4	Generated by the MAC-1G_AMBA when the CSR.26 bit is cleared.	Checked by the MAC-1G_AMBA and passed to the host.

2.14.3 Collision handling

Collision detection is performed via the *col* input pin. If a collision is detected before the end of the PREAMBLE / SFD, then the MAC-1G_AMBA completes the PREAMBLE / SFD, transmits the JAM sequence, and then initiates a backoff computation. If a collision is detected after the transmission of the PREAMBLE and SFD, but prior to 512 bits being transmitted, the MAC-1G_AMBA immediately aborts the transmission, transmits the JAM sequence, and then initiates a backoff. If a collision is detected after 4096 bits have been transmitted, the collision is termed a late collision. The MAC-1G_AMBA aborts the transmission and appends the JAM sequence. The transmit message is flushed from the FIFO. The MAC-1G_AMBA does not initiate a backoff and does not attempt to retransmit the frame when a late collision is detected.

The MAC-1G_AMBA uses a "truncated binary exponential backoff" algorithm for backoff computing, as defined in the IEEE 802.3 standard, and outlined on Figure 28. Backoff processing is performed only in a half duplex mode. In full duplex mode, collision detection is disabled.

**Figure 37. Backoff process algorithm**

2.14.4 Deferring

The deference algorithm is implemented as required by the 802.3 specification and outlined on Figure 29. The IFG (InterFrame Gap) timer starts to count whenever the link is not idle. If the activity on the link is detected during the first 60 bit times of the IFG timer, the timer is reset, and restarted once activity has stopped. During the final 36 bit times of the IFG timer, the link activity is ignored.

Carrier sensing is performed only when operating in half duplex mode. In full duplex mode, the state of the *crs* input is ignored.

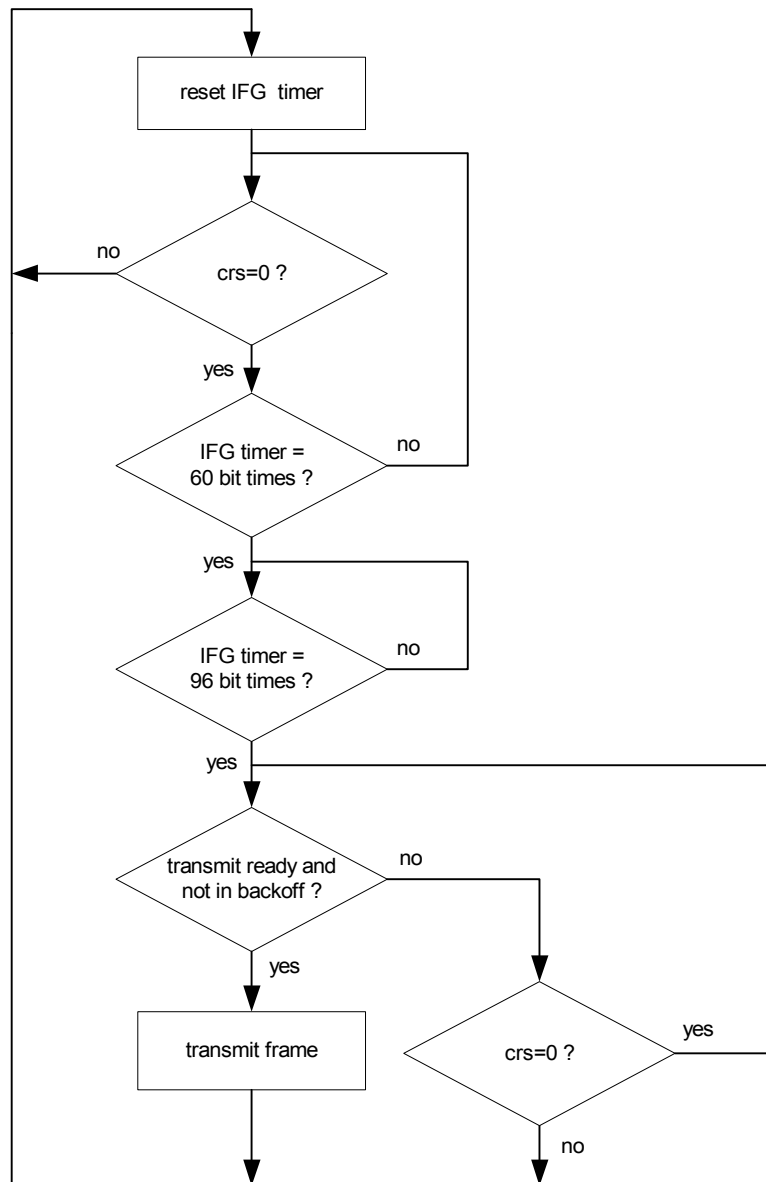


Figure 38. Deference process algorithm

2.14.5 Receive address filtering

There are three kinds of addresses on the LAN: the unicast addresses, the multicast addresses, and the broadcast addresses. If the first bit of the address (IG bit) is 0, the frame is unicast i.e., dedicated to a single station. If the first bit is 1, the frame is multicast i.e., destined for a group of stations. If the address field contains all 1's, the frame is broadcast and should be received by all stations on the LAN.

When the MAC-1G_AMBA operates in perfect filtering mode, all frames are checked against the addresses in the address filtering RAM. The unicast, multicast, and broadcast frames are treated in the same manner.

When the MAC-1G_AMBA operates in the imperfect filtering mode, the frames with the unicast addresses are checked against a single physical address. The multicast frames are checked using the 512-bit hash table. The MAC-1G_AMBA applies the standard Ethernet CRC function to the first 6 bytes of the frame, containing a destination address. The least significant 9 bits of the CRC value are used to index the table. If the indexed bit is set, the frame is accepted. If this bit is cleared, the frame is rejected. The algorithm is shown on Figure 30.

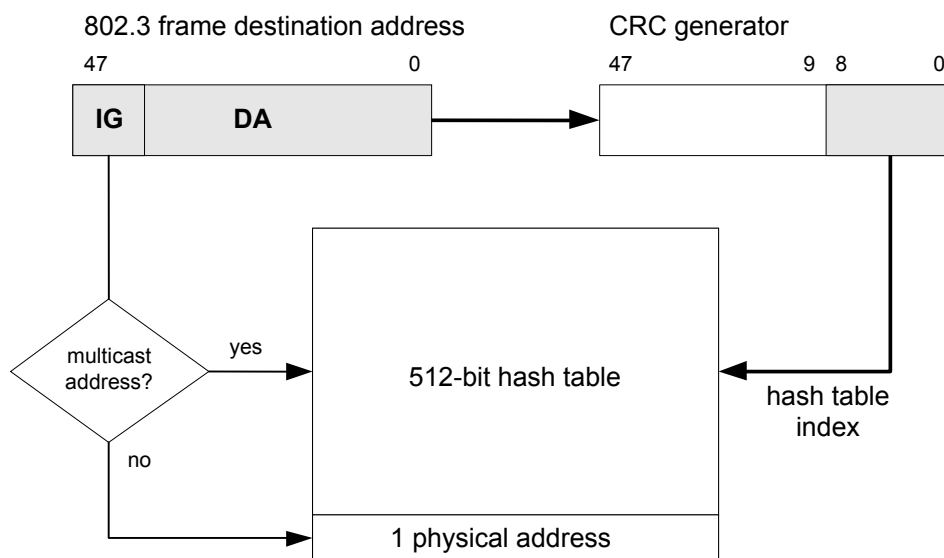


Figure 39. Filtering with 1 physical address and the hash table

It is important that one bit in the hash table can correspond to many ethernet addresses. Therefore it is possible that some frames can be accepted by the MAC-1G_AMBA, even if they are not intended to be received. This is because some frames that should not have been received have addresses that hash to the same bit in the table as one of the proper addresses. The software should perform additional address filtering to reject all such frames.

2.15 Clock control

There are four clock domains in the design:

- The TC, FCT, BP, and BD components operate synchronously with the *clkt* clock supplied by the PHY device. This is 2.5MHz clock for 10Mb operation, 25MHz for 100Mb operation, or 125MHz for 1000Mb operation.
- The RC and FCR operates synchronously with the *clkr* clock supplied by the PHY device. This is 2.5MHz clock for 10Mb operation, 25MHz for 100Mb operation, or 125MHz for 1000Mb operation.
- The TFIFO, RFIFO, TLSM, RLSM, DMA, SC, FCSTAT and MACDATA2AHB components operate synchronously with the *hclk* global clock supplied by the system.
- The CSR, MIISM, and MACCSR2APB operate synchronously with *pclk* clock supplied by the system.
- The RSTC component contains flip-flops working with all four clocks of the MAC-1G_AMBA.

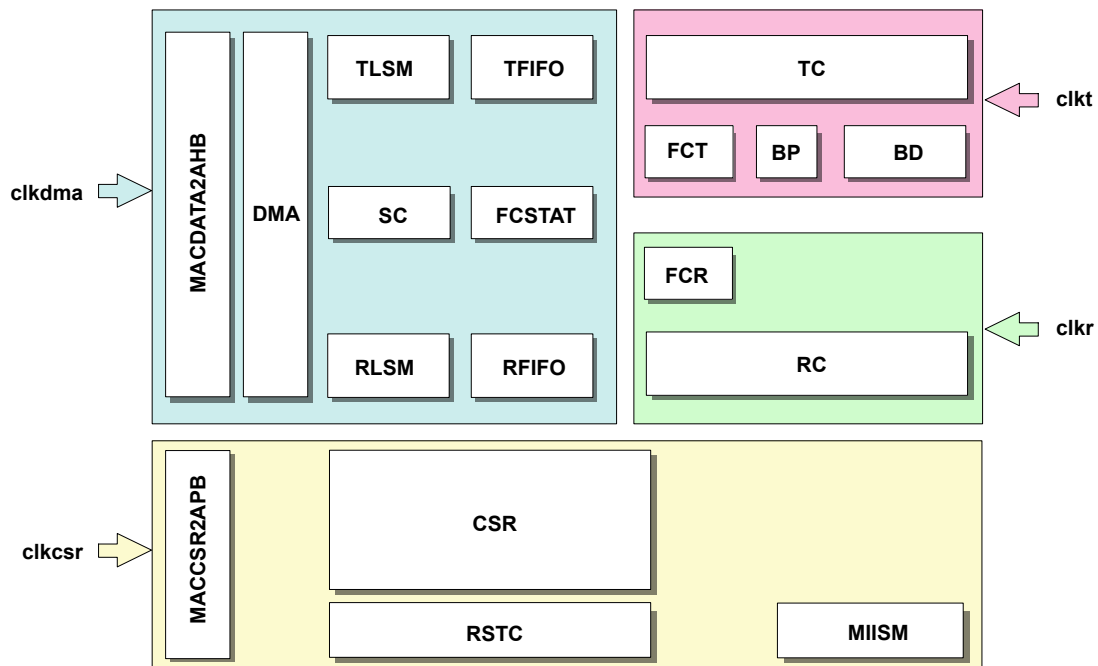


Figure 40. Clock domains

All clock signals are independent and can be asynchronous to one another.

The user should note that some minimum frequency of clock *pclk* is required for proper operation of the transmit, the receive, and the general-purpose timers. The minimum frequency for the *pclk* should be at least the *clkt* frequency divided by 64. For a proper operation of the receive timer, the *pclk* frequency should be at least the *clkr* frequency divided by 64. When the above conditions are not met, neither the transmit interrupt mitigation control, the receive interrupt mitigation control or the general purpose timer should be used. Appropriate clocks should be also supplied when the hardware reset operation is performed.

2.16 Reset control

2.16.1 Hardware reset

The MAC-1G_AMBA core contains a single input reset signal. This signal is sampled in the RSTC component during every clock signal used by the MAC-1G_AMBA. The RSTC component generates internal resets for every clock domain in the MAC-1G_AMBA.

The external reset signal should be active (high level) for at least four periods of all clocks in the design. This means that the *reset* signal timing should be adjusted according to the slowest clock used.

2.16.2 Software reset

Software reset can be performed by setting the CSR0(0) (SWR) bit. The software reset will reset all internal flip-flops.

3 User's guide

This chapter describes how to customize and use the MAC-1G_AMBA core. The MAC-1G_AMBA provides the application-specific Intellectual Property (IP) designs that are technology-independent, reusable, and easy to integrate. The MAC-1G_AMBA package provides a complete environment for you to create an ethernet network implementation that best suits your design needs.

3.1 Basic design flow

The recommended high-level design flow for the MAC-1G_AMBA is:

1. Install the MAC-1G_AMBA core.
2. Select a simulation tool for pre-synthesis (RTL) and post-synthesis (Gate-Level) verification.
3. Define your implementation. You can modify the default parameters in order to obtain the features in your implementation.
4. Set up the dual port RAM.
5. Perform a pre-synthesis (RTL) simulation on the MAC-1G_AMBA source code to verify that the design performs the desired function.
6. Select the target technology. This information is needed before you synthesize the MAC-1G_AMBA. Contact the vendor of your choice for the appropriate target technology libraries.
7. Synthesize the MAC-1G_AMBA with the synthesis tool into a technology-dependent, gate-level design. Review the results to verify if the timing and gate count meet your design constraints. If the design does not meet your goals, use the results to determine the modification you can make. This may be an iterative process and might require modifications to the original constraints.
8. After you verify the functionality, timing, and design goals of your MAC-1G_AMBA implementation, instantiate the MAC-1G_AMBA into your ASIC or FPGA design.

3.2 Compilation notes

The VHDL model follows the IEEE 1076-87 VHDL Standard.

The Verilog model follows the Verilog-1995 Standard.

3.2.1 MAC-1G compilation

The source files for the MAC-1G are in the directory MAC_1G_AMBA/src/core/mac_1g.

The model files must be compiled into the library mapped to 'MAC_1G_LIB'.

The order of compilation is as follows:

- a) Package with utilities:
MAC_1G_AMBA/src/core/mac_1g/
utility_mac_1g.vhd (does not have to be elaborated)

- b) Components for model core:
MAC_1G_AMBA/src/core/mac_1g/
tc.vhd (or tc.v)
bd.vhd (or bd.v)
rc.vhd (or rc.v)
tfifo.vhd (or tfifo.v)
rfifo.vhd (or rfifo.v)
tlsm.vhd (or tlsm.v)
rlsm.vhd (or rlsm.v)
dma.vhd (or dma.v)
csr.vhd (or csr.v)
rstc.vhd (or rstc.v)
miism.vhd (or miism.v)
dom.vhd (or dom.v)
domreq.vhd (or domreq.v)
domack.vhd (or domack.v)
sc.vhd (or sc.v)
fcr.vhd (or fcr.v)
fct.vhd (or fct.v)
bp.vhd (or bp.v)
fcstat.vhd (or fcstat.v)

- c) Model core:
MAC_1G_AMBA/src/core/mac_1g/
mac_1g.vhd (or mac_1g.v)

3.2.2 MAC2AMBA wrapper compilation

The source files for the wrapper are in the directory MAC_1G_AMBA/src/core/mac2amba. The model files must be compiled into the library mapped to 'MAC2AMBA_LIB'.

- d) Package with utilities for wrapper:
MAC_1G_AMBA/src/core/mac2amba/
Mac2amba_package.vhd (does not have to be elaborated)
- e) Components for wrapper core:
MAC_1G_AMBA/src/core/ mac2amba /
macdata2ahb.vhd (or macdata2ahb.v)
maccsr2apb.vhd (or maccsr2apb.v)
- f) Wrapper core:
MAC_1G_AMBA/src/core/mac2amba/
mac2amba.vhd (or mac2amba.v)

3.2.3 MAC-1G_AMBA model compilation

The source files for the MAC-1G_AMBA are in the directory MAC_1G_AMBA/src/core. The model files must be compiled into the library mapped to 'MAC_1G_AMBA_LIB'.

- a) MAC-1G_AMBA core:
MAC_1G_AMBA/src/core/
Mac_1g_amba.vhd (or mac_1g_amba.v)

3.2.4 TestBench compilation

The source files for the testbench are in the directory MAC_1G_AMBA/src/tb.
The model files must be compiled into the library mapped to MAC_1G_AMBA_LIB.

The order of compilation is as follows:

- a) Supplementary source code files:
MAC_1G_AMBA/src/tb/chip
 dualram.vhd (or dualram.v)
 scdram.vhd (or scdram.v)
 tris.vhd (or tris.v)
 mac_1g_chip.vhd (or mac_1g_chip.v)
- b) Environment related files:
MAC_1G_AMBA/src/tb/env
 extclock.vhd (or extclock.v)
 linkmon.vhd (or linkmon.v)
 intmon.vhd (or intmon.v)
 csrcmd.vhd (or csrcmd.v)
 datacmd.vhd (or datacmd.v)
- c) Test Bench top-level:
MAC_1G_AMBA/src/tb/
 mac_1g_tb.vhd (or mac_1g_tb.v)

3.2.5 Compilation order

A compilation run directory, correct for supplied scripts and macros is: MAC_1G_AMBA/

Compilation order of core model is found in:

MAC_1G_AMBA/tools/mti/ee_macros/
 compile.do - for MTI ModelSim EE
MAC_1G_AMBA/tools/aldec/macros/
 create_design.do - for Aldec Active HDL

3.3 Test Bench description

The Test Bench consists of the MAC_1G_CHIP instantiation and the Test Bench environment.

3.3.1 The MAC_1G_CHIP description

The MAC_1G_CHIP is a sample design that uses the MAC-1G_AMBA core and illustrates how to build and connect on-chip dual-port RAM memories and tristates.

The MAC_1G_CHIP consist of:

- | | | |
|-----------------|---------------|-----------------------------|
| ▪ U_MAC_1G_AMBA | : MAC_1G_AMBA | - The MAC-1G_AMBA core |
| ▪ U_TRANSMITRAM | : DUALRAM | - Transmit Dual port RAM |
| ▪ U_RECEIVERAM | : DUALRAM | - Receive Dual port RAM |
| ▪ U_TSCDRAM | : SCDRAM | - Transmit SC Dual Port RAM |
| ▪ U_RSCDRAM | : SCDRAM | - Receive SC Dual Port RAM |
| ▪ U_FILTRAM | : DUALRAM | - Address Dual port RAM |
| ▪ U_TRIS | : TRIS | - Tristate buffer |

3.3.2 Test Bench environment description

The Test Bench environment components:

- | | | |
|-----------------|---------------|------------------------------|
| ▪ U_AMBAARBCMD | : AMBAARBCMD | - AMBA arbiter |
| ▪ U_APB CMD | : APBCMD | - APB interface monitor |
| ▪ U_SMEMSTIMAHB | : SMEMSTIMAHB | - Shared memory model |
| ▪ U_LINKMON | : LINKMON | - GMII/MII interface monitor |
| ▪ U_INTMON | : INTMON | - Interrupt monitor |
| ▪ U_PCLK | : CLKGEN | - APB clock generator |
| ▪ U_HCLK | : CLKGEN | - AHB clock generator |
| ▪ U_CLKT | : CLKGEN | - MII transmit clock |
| ▪ U_CLKR | : CLKGEN | - MII receive clock |
| ▪ U_RSTGEN | : RSTGEN | - Reset generator |
| ▪ U_CAM | : CAM | - CAM memory model |

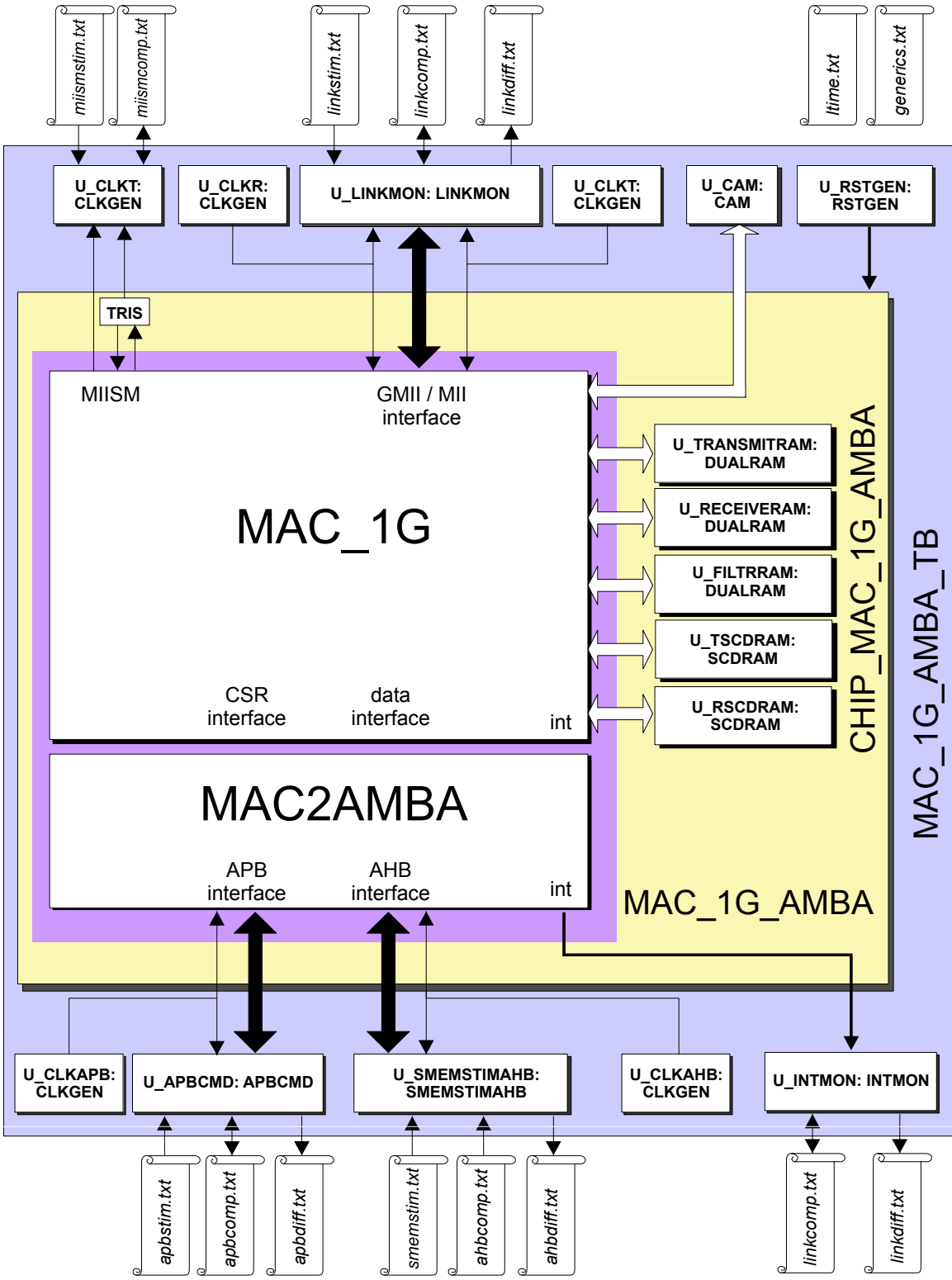


Figure 41. Test Bench block diagram

3.3.3 SMEMSTIMAHB - Shared memory model with AHB slave interface

The SMEMSTIM component is used as the stimulator for MAC AHB interface. The component simulates shared MAC-1G_AMBA and host memory (when generic parameter AHBMODE=1 or AHBMODE=2). The component stimulus file (*smemstim.txt*) describes the content of the memory during the simulation process. The format of the *smemstim.txt* file is shown below:

<cycle> <memaddress> <memdata>

where:

<cycle> - describes the clock cycle in which the data appear in the memory,

<memaddress> - describes the memory address where the data are written,

<memdata> - describes the memory data that are written.

The consecutive line cannot contain <cycle> field with value less than the value in previous line.

The SMEMSTIMAHB can operate in one of two modes: Vectors Comparator Mode (generic parameter AHBMODE=1) or Compfile Writer Mode (AHBMODE=2). The mode can be changed by a generic parameter.

The SMEMSTIMAHB component working in the Compfile Writer Mode acts as a bus analyzer. It monitors the AHB interface and writes all bus transactions into the text file (*ahbcomp.txt*). This file can be used later as the file with expected results in the Vectors Comparator Mode.

The SMEMSTIMAHB working in the Vectors Comparator Mode reads the expected results from the *ahbcomp.txt* and compares them with the actual bus transactions observed on the AHB interface. Results of the comparison are written into the file with the simulation differences (*ahbdiff.txt*).

3.3.4 APBCMD - APB interface stimulator and monitor

The APBCMD is a bus monitor for the APB interface. It can operate in one of two modes: Vectors Comparator Mode (generic parameter APBMODE=1) or Compfile Writer Mode (APBMODE=2). The mode can be changed by a generic parameter.

The APBCMD component working in the Compfile Writer Mode acts as a bus analyzer. It monitors the MAC interface and writes all bus transactions into the text file (*apbcomp.txt*). This file can be used later as the file with expected results in the Vectors Comparator Mode.

The APBCMD working in the Vectors Comparator Mode reads the expected results from the *apbcomp.txt* and compares them with the actual bus transactions observed on the MAC. Results of the comparison are written into the file with the simulation differences (*apbdiff.txt*). The stimulator file (*apbstim.txt*) is used as a set of stimulation vectors in the both operating modes. These vectors are applied to the MAC during the simulation process.

The files with the expected results, and the files with the stimulators, use a common file format. The bus transactions are described as commands on those files. The set of available commands is:

- 0 <cycles>
 - *command Wait* - waits for a specified number of the clock cycles. No bus transactions are completed when the *cmdWait* command is executed.
- 1 <address> <rdata>
 - *commandd Read* - represents a read request on the bus.
- 2 <address> <wdata>
 - *commandd Write* - Represents a write request on the bus.

3.3.5 AMBAARBCMD – AMBA™ arbiter signals stimulator

The AMBAARBCMD is an AMBA™ arbiter simulator. When generic parameter ARBMODE='1' It observes *hbusreqmac* and sets *hgrantmac* according to commands in *arbstim.txt* file. The set of available commands for that file is:

- 0 0 - *command ClearGrant* – set *hgrantmac* signal low
- 0 1 - *command SetGrant* – set *hgrantmac* signal high
- 1 0 - *command WaitBusReqLow* – wait until *hbusreqma* is low
- 1 1 - *command WaitBusReqHigh* – wait until *hbusreqma* is high
- 2 n - *command WaitNClkCycles* – wait for n hclk cycles

3.3.6 LINKMON – GMII/MII interface monitor

The LINKMON is the GMII/MII interface monitor. It can operate in one of two modes: Vector Comparator Mode, and Compfile Writer Mode. The mode can be changed by a generic parameter. The LINKMON component working in Compfile Writer Mode acts as a GMII/MII analyzer. It monitors the MAC-1G_AMBA's transmit GMII/MII pins and writes all transmit activity into the text file (*linkcomp.txt*). This file can be used later as the file with the expected results in the Vector Comparator Mode, or as a stimulus file for the MAC-1G_AMBA receive operation. The CMD working in the Vector Comparator Mode reads the expected results from the *linkcomp.txt* and compares the results with the observed transmissions on the MAC-1G_AMBA's GMII/MII interface. The results of the comparison are written into the file with the simulation differences (*linkdiff.txt*). The stimulator file (*linkstim.txt*) is used as a set of stimulation vectors for the GMII/MII interface in both operating modes. These vectors are applied to the MAC-1G_AMBA during simulation.

The format of the *linkstim.txt* and *linkcomp.txt* files is shown below:

<time> <txd/rxd> <txen/rxen> <txer/rxer> <col> <crs>

- | | |
|-------------|---|
| <time> | - The absolute simulation time. |
| <txd/rxd> | - Relative to the <i>linkcomp.txt</i> file, it indicates the value on the MAC-1G_AMBA <i>txd</i> pins. Relative to the <i>linkstim.txt</i> file, it is applied to the MAC-1G_AMBA <i>rx</i> pins. |
| <txer/rxer> | - Relative to <i>linkstim.txt</i> , this value is applied to the MAC-1G_AMBA <i>rxer</i> pin. Relative to the <i>linkcomp.txt</i> file, it reflects the value on the MAC-1G_AMBA <i>txer</i> pin. |
| <col> | - The value on the MAC-1G_AMBA <i>col</i> input. |
| <crs> | - The value on the MAC-1G_AMBA <i>crs</i> input. |

3.3.7 INT - Interrupt monitor

The INT component is the interrupt monitor. It can operate in one of two modes: Vector Comparator Mode, and Compfile Writer Mode. The mode can be changed by a generic parameter. The INT component working in the Compfile Writer Mode acts as the MAC-1G_AMBA's *int* pin analyzer. It stores all events that occurred on the *int* pin during the simulation process into the *intcomp.txt* file. In the Vector Comparator Mode all event times written to the *intcomp.txt* file, and are compared with the actual events on the MAC-1G_AMBA's *int* pin. The results of the comparison are written into the file with the simulation differences (*intdiff.txt*).

Each row of the *intcomp.txt* file contains the total simulation time of an event on the *int* pin.

3.3.8 CLKGEN – clock generation unit

The CLKGEN component is a clock generator used for controlling the *hclk*, *pclk*, *clkr*, and *clkt* signals. Period of this clock can be configured (default PERIOD=40ns).

Period of the *hclk* clock can be changed using AHBCLK_PERIOD generic parameter.

Period of the *pclk* clock can be changed using APBCLK_PERIOD generic parameter.

Period of the *clkt* and *clkr* clocks can be changed using CLKMII_PERIOD generic parameter.

3.3.9 RSTGEN – reset generation unit

The RSTGEN component is a reset generator used for controlling the *hresetn* and *presetn* reset signals. The reset is generated at the beginning of each test. The length of the reset can be configured with generic parameters RSTAHB_CYCLES for *hresetn* signal and with RSTAPB_CYCLES for *presentn* signal.

3.3.10 CAM – Content Addressable Memory model

The CAM model is a simple component used to test operation of an External address-filtering interface of the MAC-1G_AMBA. It is connected directly to the interface of the MAC-1G_AMBA.

3.4 Simulation notes

3.4.1 The simulation tool used is Model Technology.

If user want to run a chosen test case, he needs to copy all files from the test case location (eg. /tests/d32/d32_dma1) to the default (/tests/defaults) directory, then:

Step #0a – Invoke the GUI interface of MTI simulator.

Step #1a – Change Directory to your work directory (menu File/Change Directory),
for example:

./mac_1g_amba/ or x:\ mac_1g_amba

Step #2a – Enter command to execute compilation macro:

do tools/mti/ee_macros/compile.do

Step #3a – Enter command to execute simulation macro:

do tools/mti/ee_macros/simulate.do

Step #4a – Enter command to start simulation, for example:

run 100 us

Or better the value which is placed in the *time.txt* file in the test case directory, it assures that all simulation vectors will finish.

Step #5a – Enter command to end simulation:

quit -sim

As a result of simulation in the *default* directory should appear new log file *ahbdiff.txt*, *apbdiff.txt*, *intdiff.txt* and *linkdiff.txt* with detailed information, if test pass or failed and other conditions.

If user wants to run a group of test cases or the whole test set:

Step #1b – the design need to be compiled as is said in the **Steps #0a #1a #2a**,

Step #2b – run the shell script with the name of the group which needs to be verified, *08bit_tests.tcl* (or *08bit_tests.bat*) for example. Note that those shell scripts are invoking the *GROUP_NAME.tcl* file, so this file need to be present in the *tools/mti/ee_scripts* directory.

As a result of simulation in every test case location directory should appear new log files *ahbdiff.txt*, *apbdiff.txt*, *intdiff.txt* and *linkdiff.txt* with detailed information, if test pass or failed, additionally the MTI console log is put into */tools/mti/ee_reports/* directory as a summary report.

3.4.2 The simulation tool used is NC-SIM.

If user want to run a chosen test case, he needs to copy all files from the test case location (eg. /tests/d32/d32_dma1) to the default (/tests/defaults) directory, then:

Step #0a – Run the shell script *compile.sh* (*compile.bat*) from *nc_macros* directory. This is compilation macro.

Step #1a – Run the shell script *simulate.sh* (*simulate.bat*) from *nc_macros* directory. This macro invokes the GUI interface of NC-SIM simulator with Waveform Window

Step #2a – Enter command in console window to start simulation, for example:
run 100 us

or better the value which is placed in the *time.txt* file in the test case directory, it assures that all simulation vectors will finish.

Step #5a – Enter command to end simulation:
quit

As a result of simulation in the *default* directory should appear new log file *ahbdiff.txt*, *apbdiff.txt*, *intdiff.txt* and *linkdiff.txt* with detailed information, if test pass or failed and other conditions.

If user wants to run a group of test cases or the whole test set:

Step #1b – Run the shell script *compile.sh* (*compile.bat*) from *nc_scripts* directory.

Step #2b – Run *all_tests.sh* (*all_tests.bat*) for the whole test set. Note that this shell scripts are invoking the *GROUP_NAME.tcl* file, so this file need to be present in the *tools/cadence/nc_scripts* directory.

As a result of simulation in every test case location directory should appear new log files *ahbdiff.txt*, *apbdiff.txt*, *intdiff.txt* and *linkdiff.txt* with detailed information if test pass or failed, additionally the NC_SIM console log is put into *./tools/cadence/nc_reports/all_tests.log* file as a summary report.

4 Sources

- ◆ IEEE 802.3 CSMA/CD, IEEE, 2002.
- ◆ 21143 PCI/CardBus 10/100 Mb/s Ethernet LAN Controller Hardware Reference Manual, Intel, 1998.
- ◆ AMBA™ Specification Rev. 2.0

Contact:

Intel Corporation

P.O. Box 7641

Mt. Prospect, IL 60056-7641

Phone: 800-548-4725

URL: <http://www.intel.com>